

Aiding the Design of Critical Software Systems by Iterative Exploration of Distinct Requirement Violation Scenarios*

Richárd Szabó ^{1†}, Dániel Szekeres ^{1†}, Simon József Nagy^{1,2},
Zoltán Thimár², István Majzik ¹, Zoltán Micskei ¹,
András Vörös ¹

¹*Department of Artificial Intelligence and Systems Engineering,
Budapest University of Technology and Economics, Műegyetem rkp. 3.,
Budapest, H-1111, Hungary.

²thyssenkrupp Components Technology Hungary Ltd., Budafoki út 56.,
Budapest, H-1117, Hungary.

*Corresponding author(s). E-mail(s): szabor@mit.bme.hu;

Contributing authors: szekeres@mit.bme.hu; snagy@mit.bme.hu,
simon.nagy@thyssenkrupp-automotive.com;

zoltan.thimar@thyssenkrupp-automotive.com; majzik@mit.bme.hu;
micskeiz@mit.bme.hu; vori@mit.bme.hu;

[†]These authors contributed equally to this work.

Abstract

Fault-tolerant behavior is prevalent in many fields of critical software systems. The complexity of such systems necessitates rigorous techniques for ensuring correct and reliable operation. Performing design reviews by experts is the conventional method in the industry, but it is hard to substantiate the completeness of the result.

We performed a multi-year study with an automotive supplier on supporting standard-mandated safety evaluations with model checking in the context of

*This version of the article has been accepted for publication, after peer review but is not the Version of Record and does not reflect post-acceptance improvements, or any corrections. The Version of Record is available online at: <http://dx.doi.org/10.1007/s10664-025-10786-z>. Use of this Accepted Version is subject to the publisher's Accepted Manuscript terms of use <https://www.springernature.com/gp/open-research/policies/accepted-manuscript-terms>

a critical steer-by-wire function. However, existing formal verification methods returned too many redundant violations that the engineers deemed unrealistic or acceptable.

Based on these findings, we propose a new process focusing on systematic, iterative exploration of distinct requirement violation scenarios with model checkers to aid the design and verification of fault-tolerant safety-critical software systems. The main challenge lies in the need to find all essentially distinct problems of subsystems without enumerating the potentially infinitely many problematic execution paths.

The novelty of our approach is using abstract scenario descriptions constraining the model checker to unknown violations while allowing some violations to be categorized as acceptable by subject experts. Our approach has multifold benefits: It turns implicit assumptions into formalized environmental models and scenario descriptions, sheds light on hard-to-find requirement violations, and can be integrated into existing development processes.

We present the applicability of our approach in the automotive study, where it revealed previously unknown problems in the system design and gave formal proofs to help the engineers negotiate with the client regarding required modifications.

Keywords: reliability engineering, model checking, automotive, critical software systems

1 Introduction

Context

The need for fault-tolerant and fail-operational behavior is prevalent in today's safety-critical cyber-physical systems. Such modern critical systems are increasingly software-intensive, enabling complex fault tolerance logic. The automotive industry exemplifies this well, as systems providing essential vehicle functions like steering or braking must be fail-operational (Huang et al. 2019): in case of an error, the vehicle cannot always stop safely (e.g. in the middle of heavy traffic), a "limp home" functionality is necessary to safely reach a maintenance site. The prevalence of advanced driver-assistance systems in modern vehicles and the growing adoption of autonomous driving functions further exacerbates the need for highly available software-intensive subsystems inside vehicles, as they often depend on the presence of brake-by-wire and steer-by-wire features. These increasing expectations and the growth of complexity brought about by software-based solutions make uncovering weaknesses of a given design and the different scenarios where subsystem-level requirements are violated ever more challenging.

In close collaboration with thyssenkrupp Components Technology Hungary Ltd., we worked on applying model checking to aid the design of various automotive components. We observed several challenges when trying to incorporate model checkers into the safety concept phase of designing fault-tolerant automotive components with the

aforementioned expectations. Based on our experience, we developed a process more suitable to this phase and the encountered kinds of requirements.

Fault tolerance can appear on multiple levels of the system hierarchy. In a steer-by-wire system, for example, if the road wheel actuator can no longer control the vehicle because of a fault, alternative steering functions (like steering by braking (Rupanov et al. 2014)) can be activated. This is a higher-level fault tolerance measure compared to the redundancies built into the road wheel actuator component itself. As such, occasional subsystem-level requirement violations might be tolerable as higher-level fault tolerance mechanisms compensate for them. When evaluating subsystem designs, some requirement violations might be acceptable, but these must be considered when designing and verifying higher-level fault tolerance measures.

Similar challenges regarding acceptable requirement violation scenarios arise when performing reliability and availability analysis: a component might violate its requirements due to random permanent and transient faults, but instead of eliminating these violations, we only want to make sure that all combinations that can cause a system-level failure have been taken into account when constructing fault trees, FMEA tables, or other standard reliability models. This is far from trivial when faults affect the dynamic behavior of the system, e.g. when transient failures result in complex reinitialization procedures, or reconfiguration is used for fault tolerance.

This work is motivated by the need to ensure that all distinct errors and failure scenarios are taken into account when designing fault-tolerant architectures in safety-critical systems. The common way in the industry is to perform design reviews by experts, but model checkers can be great tools to aid in uncovering complex, hard-to-find scenarios that can easily elude human reviews.

Objective

Our research aimed to develop a process to aid the work of systems and safety engineers in the safety concept phase. Formal methods, specifically model checking, can aid in uncovering weaknesses of different architectures, and formalization of the system can make the implicit “mental” assumptions of the engineers become explicit and reviewable. However, most methodologies about the industrial usage of model checkers focus on verifying the final system and supporting the safety case directly by proving that the system as a whole cannot violate a given formalized requirement, resulting in several challenges for our use case.

Understanding that some requirement violations can be acceptable, our objective was to develop a process to find and categorize these violations in a systematic way. The process expects a model of the current (sub)system design, an optional set of initial assumptions about the environment, and a requirement as input. Its goal is to explore the scenarios leading to the violation of the requirement based on the design and the environmental assumptions. After analyzing such scenarios, we either want to (1) modify the design (if a realistic problematic violation scenario is present which is not handled by higher-level fault tolerance and does not have low enough probability), (2) extend the set of environmental assumptions (if a scenario is unrealistic), or (3) mark scenarios as acceptable which can be either handled on a higher level or is tolerable because of its low probability (potentially resulting in independence constraints for

the physical architecture). This decision requires the contribution of human experts with knowledge about the system design.

A part of this can be automated: a model checker can exhaustively analyze a formal model of the design and output a trace that violates a formalized description of the requirement (Baier and Katoen 2008). Such tools have already been successfully employed in industrial applications (Lahtinen et al. 2012; Kaleeswaran et al. 2023; Pontes et al. 2014; Gleirscher and Marmsoler 2020; Osaiweran et al. 2014; Miller et al. 2005). We wanted to utilize already existing model checkers without modifying their inner working, using them as information providers (similarly to how a simulator would be used, but more focused), not as direct safety proofs. Simulators could be used as an alternative, but they cannot explore the behavior of the model exhaustively and it harder to focus their exploration based on a requirement to satisfy.

Challenges

Formal verification tools can identify elusive requirement violations, but we face the following challenges when using them in the context above:

1. Early design models are often constructed at a high abstraction level, allowing unrealistic execution paths, while assumptions about realistic execution paths mostly reside in the minds of the designers.
2. We want to reveal all possible violations, but not necessarily eliminate all of them, as they might have low probability or are expected to be mitigated by higher-level fault-tolerance, but simply enumerating all problematic execution paths leads to possibly infinitely many different executions exemplifying the same problem (e.g. by only changing the ordering of independent events or changing the behavior of irrelevant components), instead of showing distinct problems. Instead of asking the model checker to enumerate all (or a bounded number of) different traces, we can tell the model checker to return a different violation after a check, but this results in the same problem: as the model checker sees only the mathematically precise model, it will distinguish traces that differ only in irrelevant details.
3. Requirements for fail-operational systems involve not only avoiding erroneous states but also making sure that (constrained) operation can continue after faults. This exacerbates the previous two: violation scenarios are often harder to describe than for error avoidance, and assumptions about the realistic behavior of the environment become more important.

Method

We present an iterative process for exploring requirement violation scenarios assisted by a model checker, extending and modifying the classic iterative method for using model checkers (Baier and Katoen 2008; Lahtinen et al. 2012; Kaleeswaran et al. 2023; Kang et al. 2013). A human expert with knowledge about the system categorizes each violation found by the model checker as either *spurious* (caused by the high level of abstraction of the design models), *acceptable*, or *unacceptable*. After finding an acceptable violation scenario, we want to make sure that the model checker does not output

it as a counterexample again without changing the system design to eliminate it. Moreover, we want to ignore *any other* counterexamples that exemplify *the same problem*, as described above. Instead of telling the model checker to ignore only that specific trace, we want to eliminate all similar traces. To do this, the human expert creates *abstract violation scenario descriptions* – representing sets of traces – to constrain the model checker to violations that are sufficiently distinct from already known ones. These descriptions are fed back to the model checker when asking for other violating traces.

Instead of aiming for full automation, the incorporation of human experts makes the process easier to integrate into industrial methodologies and comply with rigorous safety standards even when certified tools are unavailable. The output of our process is a formal model *and a tabular description of the results* in support of reviewing the results, mitigating subjectivity in the categorization verdicts and human error in translating natural language to formalized model elements.

We consider two main use cases of the information generated by the proposed process (formalized assumptions, abstract scenario descriptions for acceptable violations, and the improved architecture):

- Reliability analysis: the list of requirement violation scenarios can be used when constructing standard reliability analysis artifacts, like fault trees or FMEA tables. Our method can be integrated into existing industrial processes compliant with safety standards like *ISO 26262*.
- Designing and verifying higher-level fault tolerance measures based on the violation scenarios of each subsystem: the compiled list of requirement violation scenarios can be used to ensure that the higher-level fault tolerance measures cover all possibilities, both during design and verification activities.

This paper gives a systematic framework for the described exploration approach and presents a specific instantiation of it, mapping its elements, including the abstract violation scenario descriptions, to formal representations. We also evaluate its practical usability.

Contributions

The main contributions of our paper are the following:

1. A novel process for using model checkers to systematically *explore* the possible violating scenarios for subsystem-level fault tolerance requirements, aiming to assist the design of higher-level fault tolerance measures and reliability and availability analysis of the system.
2. An example instantiation of the process using the Gamma Statechart Composition Framework, used to show the realizability of the process.
3. A complex industrial study applying our process to a fault-tolerant, distributed sensor architecture from the automotive domain.

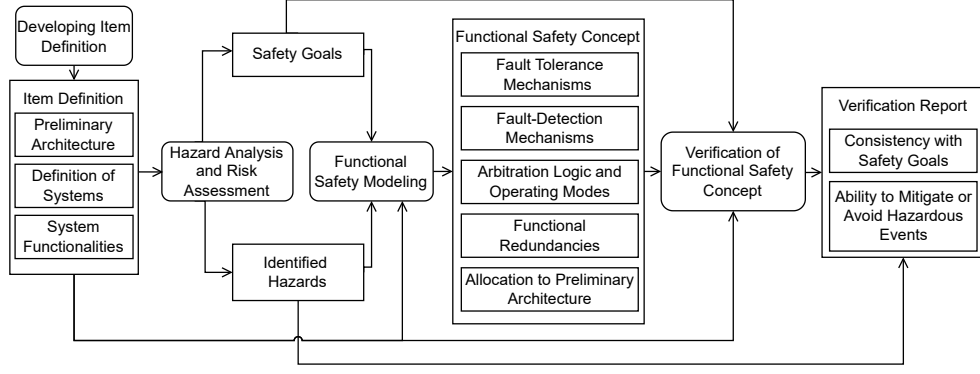


Fig. 1 Safety-related processes and work products in the concept phase of the development

The novelty of our process lies in the categorization of violation scenarios, allowing “acceptable” violations, the iterative refinement of *environmental assumptions*, and the usage of *abstract scenario descriptions*, all of which aim to be better suited to the needs of the concept phase and liveness requirements.

The Gamma-based concretization is one possible realization of the general process, which does not aim to be optimal for now. It was created as a proof of concept for the applicability of the process, and we plan to enhance the user experience of the tooling based on the industrial findings and feedback.

We use model checkers as black boxes, focusing on improving the efficiency of the interaction between the user and the tool rather than the internal performance of the tool. Although state-of-the-art model checkers could now generate hundreds of violating runs even for industrial models, efficiently producing numerous violations alone is insufficient for practical usability. Therefore, instead of gauging their computational complexity or scalability, we evaluate practical usability through a qualitative study in an industrial context.

Results and Impact

The application of the proposed process on the presented system architecture gave formal proof to the system designers of their hypothesized violation scenarios and showed additional problems in the system design. The revealed problems and the formal proofs provided a strong basis for the engineers in the negotiations with the OEM regarding the required modification and further development. Incorporating the proposed exploration method into industrial design processes for fault tolerant systems can aid in earlier recognition of design flaws and even highlight problems that could have been missed and provide an exact list of weak points of the design, which must be covered by safety mechanisms to improve the system.

2 Background

2.1 Safety Requirements in Automotive Development

Reliability, availability, and safety are primary concerns in the automotive industry. The malfunction of automotive systems can cause catastrophic events, such as traffic accidents. State-of-the-art automotive systems such as steer- and brake-by-wire systems have numerous high-complexity software and hardware solutions to mitigate the effect of hardware, software, and communication faults. As a result, automotive systems implement several advanced safety measures. The development process is specified by the *ISO 26262* (ISO 2011) functional safety standard, which requires the specification and verification of safety measures already in the concept phase of the development. Figure 1 depicts the safety-related activities and artifacts of the concept phase required by *ISO 26262*.

- The *item definition* is the vehicle level functionality of the item, including its requirements, interaction with the driver and the environment, as well as other items of the vehicle. In addition, it may include the definition of the preliminary architecture of the item to support easier execution of subsequent activities of the development
- *Hazard analysis and risk assessment* identifies and categorizes the hazards that malfunctions in the item can trigger and formulates the safety goals related to the prevention or mitigation of the hazardous events.
- The *safety goals* define the vehicle-level safety requirements, specifying all hazards that must be mitigated or prevented to avoid unreasonable risk.
- The *functional safety concept* (FSC) includes the high-level specification of safety measures and safety mechanisms, such as diagnostic and control mechanisms. In a fault-tolerant system, the FSC may define for example that if a subsystem fails, another component shall use arbitration of control and reconfiguration to reach a safe state where the functionalities can be provided without the faulty subsystems.
- The *verification report* includes evidence that the FSC is consistent with the safety goals and that hazardous situations, such as hardware failures, are mitigated or avoided by fault-detection and -tolerance mechanisms.

To specify the importance of the safety requirements and safety goals, the Automotive Safety Integrity Levels (ASIL) are defined in the standard. The system and the subsystems in the designed architecture shall comply with the safety requirements and safety goals at their respective ASILs, with ASIL D representing the most stringent and ASIL A representing the least stringent level.

The conformance of the system architecture to the safety goals must be verified using inductive and deductive analysis techniques according to *ISO 26262*. Inductive safety analysis collects all the component- and part-level failure modes and evaluates their effects on the higher-level subsystems and the system itself, while deductive safety analysis techniques search for the possible causes of the system-level malfunctions by refining the failure propagation models.

In the automotive industry, Failure Modes and Effects Analysis (FMEA) is used for inductive safety analysis to evaluate the system-level effect of part-level failures, and Fault Tree Analysis (FTA) is used for deductive safety analysis.

Currently, constructing fault trees and FMEA tables relies on manually inspecting the system design and evaluating the effects of component faults. The results are verified using manual reviews and cross-checking the inductive and deductive analyses. This is prone to miss complex violations caused by the interaction of permanent and transient failures of communicating components. To improve the efficiency and reliability of this construction process, we propose the incorporation of an iterative sub-process that uses a model checker to aid in exploring all possible violations of a requirement.

2.2 Formal Modeling of Complex System Architectures

Designing complex, software-intensive systems is a well-known challenge in the automotive industry, since modern vehicles can contain several hundred Electronic Controller Units (ECUs). These ECUs must operate together, in many cases in a fail-operational way. Model-based systems engineering (MBSE) is widely used in the automotive industry to enable the usage of verification, test generation, and code generation techniques to prove the correctness of the system behavior and improve the overall quality of the system. However, the use of MBSE is not enough to solve the problems in the automotive industry by itself, it must be combined with component-based approaches, which enable the engineers to solve smaller problems in each step and build systems from smaller subsystems, which are most of the time some variant of already existing subsystems ([Lee and Sangiovanni-Vincentelli 2011](#)).

Several widely used modeling languages are available for defining the architecture and behavior of a system, both higher-level (e.g., UML, SysML, AADL) and lower-level (e.g., UPPAAL) ones, each of them with a different level of precision regarding its semantics. To use mathematical tools for reasoning about a system design, a formal model of the system is needed with mathematically precise semantics ([Baier and Katoen 2008](#)). For the process described in this paper, being able to describe both the structure of the system and the dynamic behavior of each component is also necessary.

High-level modeling languages are often analyzed by first translating them to lower-level languages and mathematical formalisms. For example, the Gamma Statechart Composition Framework ([Molnár et al. 2018](#)) can interpret a (subset of) the SysML modeling language (although SysML has some holes in its semantics, Gamma defines precise semantics for those parts as well) by transforming them to Gamma statecharts automatically, then transforming them to the low-level formalism of the chosen model checking backend (UPPAAL, SPIN, NuXmv or Theta XSTS).

2.3 Formal Description of System Requirements

Automated formal analysis needs the requirements to be formulated in a mathematically precise way. Invariant properties checked by analyzing the reachable states are widely used to specify error avoidance requirements – however, liveness (the ability to

reach a state infinitely often or constantly stay in it after an initial period) and stabilization (meaning that after a disturbance, the system must be able to return to its nominal behavior following a transient) requirements are also prevalent, especially for fault-tolerant systems. Such requirements define constraints for the entire trace, needing a more expressive requirement description language to define formally. Temporal logics, like *linear temporal logic (LTL)* (Baier and Katoen 2008), provide a formalism to describe such properties with mathematical precision and well-defined semantics.

LTL defines the following temporal operators:

- $\mathbf{X}p$: p is true in the next state
- $\mathbf{F}p$: p is true in some future (including current) state
- $\mathbf{G}p$: p is true in every future (including current) state
- $p\mathbf{U}q$: q is true in some future state, and up until then starting from the current state, p is true in every state

p and q can be Boolean formulae over state variables or nested temporal formulae. See e.g. Principles of Model Checking (Baier and Katoen 2008) for a formal definition.

Although we use LTL specifically in our case study, the method can be paired with other linear-time temporal logics like Metric Temporal Logic or Timed Linear Temporal Logic.

2.4 Model Checking

Simulation on its own can often lead to missing elusive requirement violation scenarios, especially when the model has an infinite set of traces (e.g. because of non-deterministic loops or numerical inputs). Model checking (Baier and Katoen 2008) is a verification technique that explores all possible execution paths in the system model, either giving a counterexample for the given requirement or showing that the system truly satisfies it. As exploring the large state spaces arising in practical system models is a severe computational challenge, various model-checking algorithms have been proposed in the literature (Jhala and Majumdar 2009), each of them tailored to different kinds of systems with different model checker tools implementing them (Holzmann 1997; Cimatti et al. 2002; Cavada et al. 2014; Tóth et al. 2017).

Aiming for exhaustivity leads to a high computational demand, especially for complex models. Consequently, applying model checking to the entire detailed system model is often infeasible. Instead, it can be more convenient to verify isolated subsystem models and utilize the results as assumptions when verifying system-level properties. Still, it has already found successful applications in multiple industrial domains (Lahtinen et al. 2012; Kaleeswaran et al. 2023; Pontes et al. 2014; Gleirscher and Marmsoler 2020; Osaiweran et al. 2014; Miller et al. 2005).

Critical requirements for fail-operational systems often involve liveness properties and complex temporal behaviors (e.g. stabilization after any transient error), describing what desirable operation *must eventually* happen after a finite amount of time or even *repeat infinitely often* in any run. Formalizing such properties can be done using temporal logics, like LTL. Unlike for requirements about avoiding error states, where different problems are often identified by different error *states*, problems for complex

temporal properties are often shown only by whole *traces*. Because of this, exploring the *reachable state space*, focusing only on which states are reachable at all, is not enough – we are interested in the *set of possible traces* of the model.

Inputs and outputs

Model checkers require a formal description of the system and the requirement as inputs. They output a verdict about the satisfaction of the requirement, and typically a counterexample in case of violation in the form of a violation trace: an execution path violating the requirement. Counterexamples for LTL properties are often infinitely long traces with a lasso shape: an initial part that happens only once, followed by an infinite loop. Such counterexamples still have finite descriptions.

Need for multiple distinct counterexamples

When considering quantitative properties such as reliability and availability, or assessing the limitations of a component in a system with multiple levels of fault tolerance, the aim is not to eliminate all requirement violations in the model. The challenge of utilizing model checking in reliability engineering and designing fault tolerance concepts is that even if a single violating trace exists, it reports that trace as a counterexample and stops. As we do not always want to modify the system architecture because of the existence of a single violating trace, but we still want to know whether *other* violations exist or not, we need to make the model checker report other violations that we do not know yet. Additionally, we do not simply seek *any other* trace, but a trace that exemplifies a *different problem*. This paper describes a systematic way to do this.

Classical iterative approach to model checking

[Baier and Katoen \(2008\)](#) defines the basic model checking process as an iterative activity with the following steps:

1. Create a formal model of the system.
2. Formalize the requirement of interest using a property specification language.
3. Run the model checker.
4. (a) If the property holds, move on to the next property if any.
 (b) If it does not, then analyze the counterexample, refine either the design (in case of a real problem), the model (if it does not reflect the real design appropriately), or the property (if it does not reflect the real requirement appropriately), and start the process again.

This iterative approach has been successfully utilized in the industry to aid verification activities ([Lahtinen et al. 2012](#); [Kaleeswaran et al. 2023](#); [Kang et al. 2013](#)). In the classic setting, model checking is mostly used as a direct proof that the design (or at least its formal model) completely satisfies a formalized requirement. In this view, no violation is acceptable, and all of them must be eliminated. Although this iterative approach recognizes that a violation is not always a real problem of the design, it always considers the violation to be caused by something being wrong which must be changed: if not the design, then the formalization of it (so the model must be changed

to better reflect the real system), or the formalized property (so it must be changed *to better reflect what is meant by the original textual requirement*). This makes the classical view of how model checkers are used hard to apply to use cases when the goal is not to eliminate all violations instantly.

Seemingly, the simplest way to allow filtering the violations is to either ask the model checker for multiple violation traces, or equivalently – if it does not support this – to automatically feed the already found violations to the model checker (e.g. as a part of the property to verify), forbidding the generation of a known acceptable violation trace again. Modern model checking algorithms have become performant enough that it is possible to get hundreds of different violating traces in an acceptable amount of time for practically useful models, but their output must still be processed by humans when the property of interest does not hold. One problem with this is that, as the trace set can be infinite even for finite models, asking for a huge number of different traces might still miss some problematic scenarios. Another one is that human experts have to analyze all of these traces, many of which can exemplify the same problem, so no new information is gained from their analysis.

These problems call for new approaches to integrate model checking into development processes.

3 Description of the Automotive Industrial Context

This work was performed in an industrial context with close collaborations of systems and safety engineers. This section presents the details of the industrial site and system studied: thyssenkrupp Components Technology Hungary Ltd. and the steering wheel angle measurement functionality of a steer-by-wire (SbW) system.

3.1 thyssenkrupp: the industrial site and the collaboration

As a subsidiary of thyssenkrupp Automotive Technology, thyssenkrupp Components Technology Hungary Ltd. has been developing electronic steering systems for passenger cars since 1999. Thyssenkrupp’s Budapest office designs and develops the hardware and software of safety-critical electromechanics steering systems: starting from the unique hardware blocks to the full application software, everything is custom-made in Budapest, Hungary. The office employs around 800 people, and its electronic power steering system is used by major OEMs. The development process adheres to state-of-the-art standards (e.g., *ISO 26262* for safety, AUTOSAR for software architecture). The company is an early adopter of model-based systems engineering (MBSE) practices, developing advanced internal tooling and methodologies. These characteristics made the site a good candidate for studying the application of formal verification.

Researchers from Budapest University of Technology and Economics (BME) have long collaborated with thyssenkrupp on various systems engineering topics. The study reported in this paper was performed in the context of a nationally-funded project between 2021 and 2024 (2019-1.3.1-KK-2019-00004, Safety Science and Technology Competence Center) with the following main phases and milestones:

- 2021 Focus group meetings and workshops with appr. 10 senior systems and safety engineers, collecting the current practices and challenges. The engineers were working in different departments, some of them were individual contributors while others team leaders, and had significant experience in safety-critical development (typically 5–20 years of experience). The researchers became familiar with the details of the company’s SysML-based system design methodology and recommended modifications to express subsystem functionality and allocation better.
- 2022 To study the safety process in more detail and recommend further improvements, a reasonably complex functionality and related subsystems were chosen from an ongoing development at the company. The researchers created new SysML and formal models for an absolute steering wheel angle measurement functionality (ASWA). The new models were periodically reviewed by the company’s engineers, and they clarified questions that arose during modeling.
- 2023 H1 First, formal models in low-level timed automata were created for the ASWA functionality. As the models got too complex for handling all variants of the functionality, the Gamma Framework was chosen as a modeling environment as it can generate formal models for several model checker backends. Initial counterexamples found by the model checkers (i.e., violations of safety requirements) were shared with the engineers of the company. However, during the discussions, many violations were deemed unrealistic, as they arose from unspecified environmental assumptions, while many others were acceptable due to low probability or higher-level fault tolerance measures. In order to uncover actually problematic scenarios, the model checker was configured to return more violations, a high number of which turned out to exemplify the same (potentially acceptable) weakness. Examining violations required considerable effort from both parties. Therefore, a new approach was developed to iteratively explore relevant, distinct violation scenarios.
- 2023 H2 The new approach was implemented in a specific toolchain using Gamma and SPIN as a model checker backend, which made it possible to explore numerous configurations for the functionality and a large number of traces automatically. A significantly reduced number of distinct violation scenarios was discussed with subject experts in the company, and based on the results, modifications were proposed to the system architecture in development.

3.2 Steering wheel angle measurement in a SbW system

In a steer-by-wire system, there is no mechanical connection between the steering wheel and the road wheels. As a result, a SbW system measures the angle of the steering wheel with multiple sensors to provide steering functionality. There are four subsystems in the SbW system: two *Feedback Actuator (FBA)* and two *Road Wheel Actuator (RWA)* subsystems, seen in [Figure 2](#). The RWAs are located near the rack and turn the road wheels following the changes in the measured steering wheel angle. The FBAs contain actuators near the steering wheel to simulate the perception of mechanical steering for the driver by generating feedback torque. The RWA and FBA subsystems each contain an Electronic Controller Unit (ECU). The FBA and the RWA

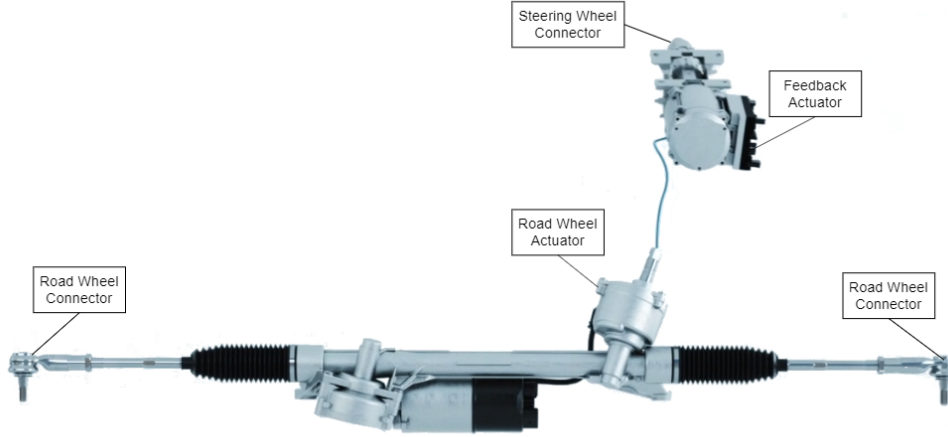


Fig. 2 The steer-by-wire system case study

subsystems are connected via two private communication buses and the two RWA subsystems are connected via an inter-MCU communication (IMC) channel.

3.3 Requirements of the Absolute Steering Wheel Angle Measurement

The integrity and reliability of the Absolute Steering Wheel Angle (ASWA) Measurement functionality are critical for vehicle-level functions relying on it, such as road wheel control and feedback generation, affecting the overall safety of the vehicle. Its loss makes the vehicle unsteerable, and incorrect steering wheel angle measurements may cause incorrect steering and feedback. As such, the system designers defined numerous safety and fault tolerance requirements in accordance with preliminary customer expectations and automotive standard *ISO 26262*. From the safety goals and vehicle-level functional requirements, they derived the following safety requirement with *ASIL D* safety criticality:

The ASWA Measurement function shall remain operational and correct even after one or two random permanent hardware faults.

3.4 Architecture of the ASWA Measurement Function

As shown in [Figure 3](#), the ASWA Measurement has four independent channels measuring the ASWA simultaneously.

The two channels allocated to the RWAs measure the ASWA directly using the dedicated *Absolute Steering Wheel Angle Sensors (ASAS)*. Within the ASAS, two gears called *main* and *multi-turn* gears are connected to the column of the steering

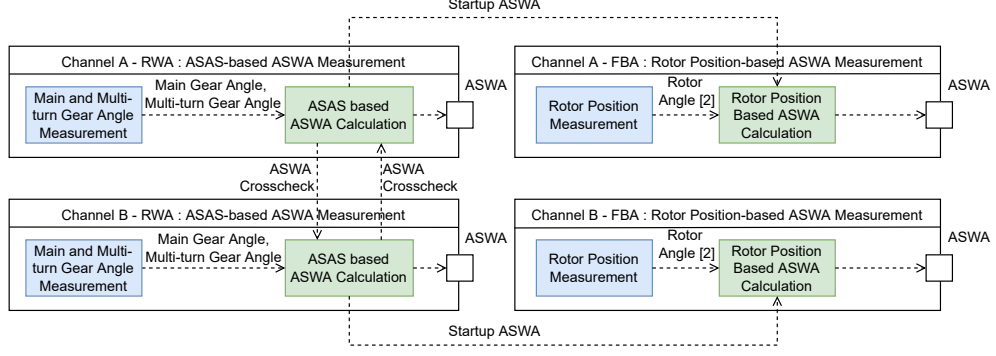


Fig. 3 Functional architecture of the ASWA Measurement

wheel, and the sensor measures the angles of these two gears. The gear ratios between the steering wheel and the two gears are different, so the ASWA can be calculated from their angles using the Vernier principle.

The two channels allocated to the FBAs use the built-in rotor angle sensor of the feedback motor. During an end-to-end steering maneuver, the steering wheel rotates several times, and during a single steering wheel rotation, the rotor of the feedback actuator also turns several times. Because of this, measuring ASWA based on the rotor position requires initialization to an already known ASWA, following which the ASWA can be calculated incrementally from the rotor angle changes over time.

3.5 Fault-tolerance of the ASWA Measurement function

The ASWA measurement in the SbW system must be resilient against the following hardware, software, and communication faults:

- F1 A hardware fault within an ECU may make its functions unavailable or the communication channels inaccessible.
- F2 Hardware faults of the ASAS and rotor angle sensor components can make corresponding angle measurements unavailable or incorrect.
- F3 Transient fault of an ECU may cause the restart of subsystems. In this case, all functions that are allocated to the restarted subsystem lose all data not stored in persistent memory.
- F4 Transient communication faults may make the private bus and the IMC channels temporarily unavailable.

Incorrect ASAS and rotor sensor data can result in incorrect ASWA Measurements, so all measurement channels have internal diagnostic mechanisms:

- The two rotor position-based channels contain redundant rotor angle sensors and each contains crosscheck mechanisms between the redundant sensors with *ASIL D* safety criticality to detect sensor failures. The crosscheck algorithms

are defined by the safety manual of the sensor components, and verified by the manufacturer. At startup, the Rotor Position-based ASWA Calculation functions require a high-integrity ASWA position from the ASAS-based ASWA Calculation functions.

- The ASAS-based channels use multiple internal crosscheck mechanisms. At startup, these channels execute a crosscheck to generate a high-integrity ASWA value for startup calibration, which is also sent to the Rotor Position-Based ASWA Calculation functions. After the calibration phase, the two ASAS-based channels can operate independently with high integrity.

4 General Process for Exploring Requirement Violation Scenarios

4.1 Aims and context

We aid the exploration of possible requirement violation scenarios for a given subsystem by utilizing a model checker to find complex violation scenarios that are hard to find manually. For instance, in our case study model, we can ask the following questions (originating from the stakeholders):

- Can we be sure that the system can reinitialize after fixing a transient failure if at least one of the RWAs and one of the FBAs have not failed permanently?
- Is the subsystem resilient against any single permanent failure if transient failures are to be expected?
- Are additional communication channels needed?

Having complex initialization processes involving multiple cross-checks and calibrations to complete after a reset makes these non-trivial to answer.

A qualitative requirement is given, but its occasional violation by the component under analysis (which is a subsystem of a composite system) can be tolerated, as some violations can be handled by a higher-level controller orchestrating fault tolerance on the system level. As such, we are not satisfied with receiving only a single violating trace from the model checker, as we want to decide for *each* possible violation whether it is acceptable. For example, in our SbW case study, if the RWA loses the ASWA signal for some reason but the FBA is still available, we can route its ASWA signal to an alternative steering function such as steering by braking.

Similarly, when considering quantitative reliability and availability analysis, we do not want to modify the system architecture because of a sufficiently low-probability violating trace, but still want to know if *other* violations exist. Thus, we must make the model checker report other violations we do not already know.

For example, we might know that the permanent failure of both RWA microcontrollers will make the RWA function unavailable, which will be taken into account when building fault trees, but can the permanent failure of one FBA and one RWA also cause a problem? Do we need to take this combination into account? During the safety concept phase of designing a system, such information is important when

enumerating independence and separation requirements for the physical architecture: a specific combination of multiple component failures might have a sufficiently low probability, but only if no common cause failure exists for these components.

To query another violation trace after getting an acceptable one from the model checker, we could restrict the model or the property to exclude exactly the previously seen trace. However, this would give possibly infinitely many traces differing only in irrelevant details. Consider a requirement violation scenario where the system cannot reinitialize if the RWA gets a transient failure, and then the FBA gets a transient failure before the RWA resets. This can be extended arbitrarily with steps of other components or transient failure-repair loops. However, these traces would be equivalent w.r.t. the problem they exemplify.

Thus we want to exclude whole “equivalence classes” of violation traces when asking the model checker for another violation. Our proposed process focuses on systematically finding all violation scenarios while addressing this challenge through *abstract violation scenario descriptions*.

We present the process in the context of an *ISO 26262*-based safety concept phase as an example of integration into existing design processes. The method itself is not limited to this standard. [Figure 5](#) shows the connection between the presented process and the *ISO 26262*-based safety concept phase.

The process expects the following inputs:

- Initial architecture concept and error model
- Component-level requirement

and produces the following outputs:

- Categorized list of violation scenarios
- Abstract scenario descriptions for the acceptable violation scenarios
- Improved architecture, which no longer exhibits the unacceptable scenarios
- Environment model and/or refined requirement formalizing assumptions about realistic execution paths

Now we describe each step of our process. [Figure 4](#) shows an overview.

4.2 Steps of the process

Step 1: Artifact preparation

We update the following information sources throughout the process from which all other artifacts originate:

- The *(sub)system architecture* defining the structure and behavior of the (sub)system to analyze, including its error model. This comes from the *preliminary system architecture* and the current *functional safety concept*.
- A *set of environmental assumptions* to rule out unrealistic execution paths like waiting for inputs indefinitely. It is mainly based on the *item definition* and potentially the *hazard analysis and risk assessment*. While an unconstrained

environment is often useful when the model is used to verify safety properties (i.e. avoiding error states) (Lahtinen et al. 2012), it can easily result in spurious violations for liveness and more complex temporal properties.

- A *set of known violation descriptions*, each of them covering a set of violation traces, as actions unrelated to the violation they describe are abstracted away. It has a two-way interaction with *hazard analysis and risk assessment*: an initial set can be created based on earlier HARA results, and the HARA can later be refined based on newly found scenarios.
- The component-level *requirement* to analyse. It is derived from the *safety goals* using the *preliminary system architecture*. While the previous items dynamically change throughout our iterative process, the requirement is static.

The initial system architecture comes from the current system design iteration. The set of environmental assumptions can be initially empty, or we can already have some assumptions needed for realistic execution paths. The set of known violation descriptions can also be initially empty or it can include trivial cases (e.g. permanent failure of all components). The abstract scenario *descriptions* can take many forms. In our case study, we will use statecharts.

Step 2: Formalization

Two formalized artifacts are derived from the sources:

- A *temporal logic formula* describing the requirement with mathematical precision.

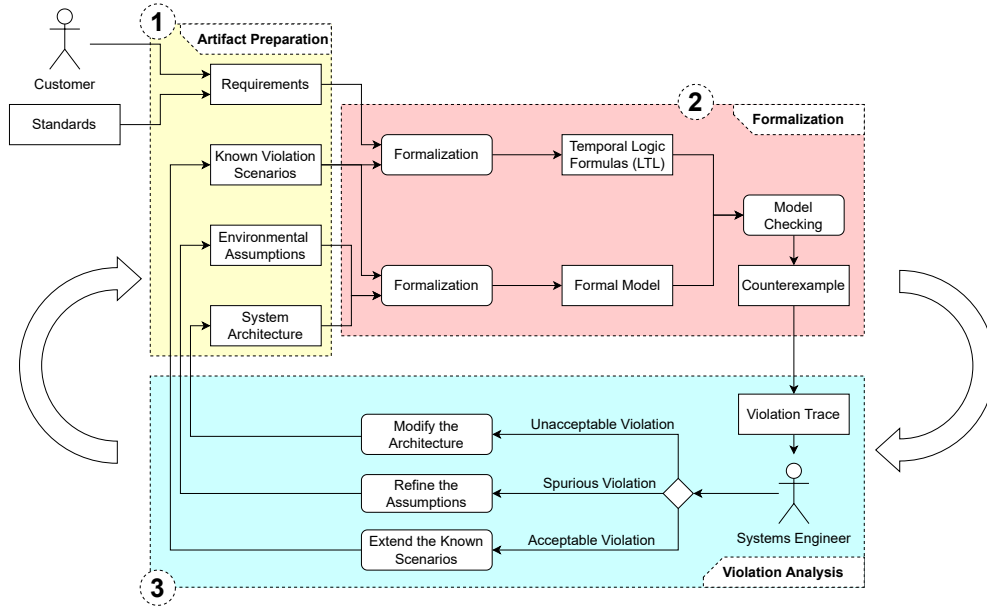


Fig. 4 Overview of the proposed process

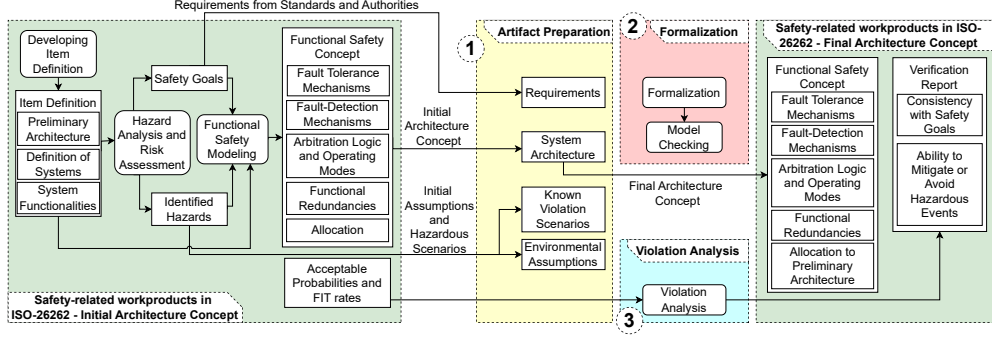


Fig. 5 Connection of the proposed process and the ISO 26262-based safety concept phase

- A *formal model* describing the behavior of the system and the assumptions about the environment. As a model cannot describe all aspects of a real system (particularly as not every property of the components is known at design time), this model inevitably contains some conservative abstraction.

The known violation descriptions can be incorporated into any of these to restrict the model checker to unknown violations.

Given an LTL formula ϕ for the requirement and a set of formulae ψ_i , each describing one of the known abstract violation scenarios, we give the formula $\neg(\psi_1 \vee \psi_2 \vee \dots \vee \psi_n) \implies \phi$ to the model checker if we want to encode them in the property.

Another option is to convert the scenario descriptions into automata that accept action sequences matching the scenario description and extend the model with these automata. The property must also be extended similarly: the left side of the implication must specify that none of these automata can accept the trace.

Counterexamples for LTL properties are either finite or lasso-shaped infinite traces (assuming a finite state space), starting with a part that happens only once before entering an infinite loop. As such, we must be able to describe both finite and lasso-shaped abstract violation scenarios. Thus, when using automata as violation scenario descriptions, we want to use a *Büchi acceptance criterion*: the automaton accepts an infinite trace if the trace enters an accepting state of the automaton infinitely many times. We specify a set of accepting states and use the following LTL template: $\neg(\mathbf{GF}(\text{accept}_1) \vee \mathbf{GF}(\text{accept}_2) \vee \dots \vee \mathbf{GF}(\text{accept}_n)) \implies \phi$, where accept_i means the automaton describing the i th scenario is in an accepting state.

A model checker is used to check whether the modified formal model satisfies the modified property. If it does, then all violation scenarios have been covered by our descriptions. If not, the model checker gives us a new violation trace.

Step 3: Violation Analysis

The counterexample trace is then interpreted and analyzed by a human expert with the appropriate knowledge about the system.

- If the trace is deemed *spurious*, then either the formal model itself must be changed as it does not describe the real system close enough, or the set of environmental assumptions is refined.
- If the scenario exemplified by this trace is an *unacceptable* violation, then the system architecture must be modified to rule it out.
- If the violation is deemed *acceptable*, either because it will be taken care of by a higher-level controller or because it has a low enough probability, then the trace is reduced to its most important parts, irrelevant actions are abstracted away, resulting in a new violation scenario description.

A possible categorization process can be seen on [Figure 6](#), note that the actual process must be tailored for the given company and domain. The presented categorization process is based on the following decisions:

1. **Can the real system exhibit the behavior described by the counterexample trace?** The Systems Engineer, using his expertise and the existing (sub)system specifications and technical specifications, and consulting with the customer, must decide whether the described behavior is a behavior of the real system. If it is not, then the trace is deemed *spurious* and we need to refine the system model.
2. **Can the real environment of the system exhibit the behavior described by the counterexample trace?** The environment of an automotive vehicle, especially those with advanced driver-assistance systems (ADAS), can contain several parameters that are difficult to model with classical reactive system models, and it is difficult to account for all the possible combinations of environmental events. Modeling only the relevant environmental events and parameters produces easier-to-follow environmental models, but results in oversimplified models in the beginning. The Systems Engineer, using his expertise and the existing environment, the system context, and interface specifications, and consulting with the customer, must decide whether the described behavior is a behavior of the environment of the system. If it is not, then the trace is deemed *spurious* and we need to refine the environment model.
3. **Does the counterexample lead to a hazardous state?** At the beginning of creating the FSC, Safety Engineers identify the hazardous states of the system. Some counterexamples that violate the requirement do not lead to a hazardous state, but lower the availability of the system. E.g.: if the ASWA Measurement Function cannot initialize at the vehicle's ignition, then the vehicle cannot start – this is a safe state of the system, but as it does not satisfy the safety-related liveness requirement of always being able to recover and reach a functioning state, it will be returned as a violation when checking that property. If the violation trace does not lead to a hazardous state, the counterexample is deemed *acceptable*.
4. **Is the counterexample covered by higher-level fault-tolerance logic?** The FSC contains the definition of *Fault Tolerance Mechanisms*, *Arbitration Logic*, and *Functional Redundancies* present in the system, which can be used to mitigate the failure of the given subsystem, before it causes system-level failure.

If the behavior described by the violation trace can be mitigated by a component that is out of the scope of the current formal model, then the counterexample is deemed *acceptable*.

5. **Is the probability of the counterexample sufficiently low?** In safety-critical systems, all fault combinations can occur with a non-zero probability when considering random faults of hardware components in the model. As a result, we cannot guarantee that a given failure cannot occur, yet we can decrease the probability of system failure by applying fault-tolerance mechanisms and safety measures until the failure probability is under a given limit. The ISO 26262-8:2018 Annex B and C require engineers to analyze only those faults and fault sequences that can significantly increase the failure rate of safety goal violations. For instance, hardware faults, which can cause safety goal violations in only n -long fault sequences where $n > 2$, can be handled as safe faults. Moreover, by decomposing the system functions into independent and redundant subfunctions and subsystems, the ASIL of the subsystems can be decreased in accordance with the rules of ASIL decomposition. Several industrial standards, such as MIL-HDBK-217F, JEP 122H, and IEC 61709 define ways to estimate the probability of random hardware faults using physics-of-failure (PoF) models and the environmental and usage conditions of the component. Using IEC 61508:2000-7 Annex D, the probability of transient software faults can be estimated as well.

Deeming a violation acceptable for probabilistic reasons can often happen when two components that are supposed to be physically and logically independent must fail at the same time (or in a very short time interval) for the violation to happen – as the probability of this is close to zero, it can be deemed acceptable, but the output documentation must contain that the acceptability is based on the assumption of independence, which must be ensured in the detailed technical system definition and the implementation.

Several aspects of the system behavior can be exploited when performing the abstraction. Some examples are:

- *Symmetry*: if the architecture is symmetric, the “mirrored” versions of a trace are essentially the same violation. An example in our case study is swapping the A and B sides in a trace: if the failure of FBA A and RWA B leads to a violation, then that of FBA B and RWA A does as well. The symmetry of two components halves the number of traces to check and may lead to exponential reduction if multiple symmetries are identified.
- *Irrelevant components*: The requirement violation is often caused by a small subset of the components. The nominal behavior of others can still be present in the counterexample, although they are irrelevant. Depending on the flexibility of scheduling in the model, this can lead to several (potentially infinitely many) concrete counterexamples that exemplify the same problem by scheduling irrelevant components arbitrarily many times before the important ones exhibit the problematic behavior.

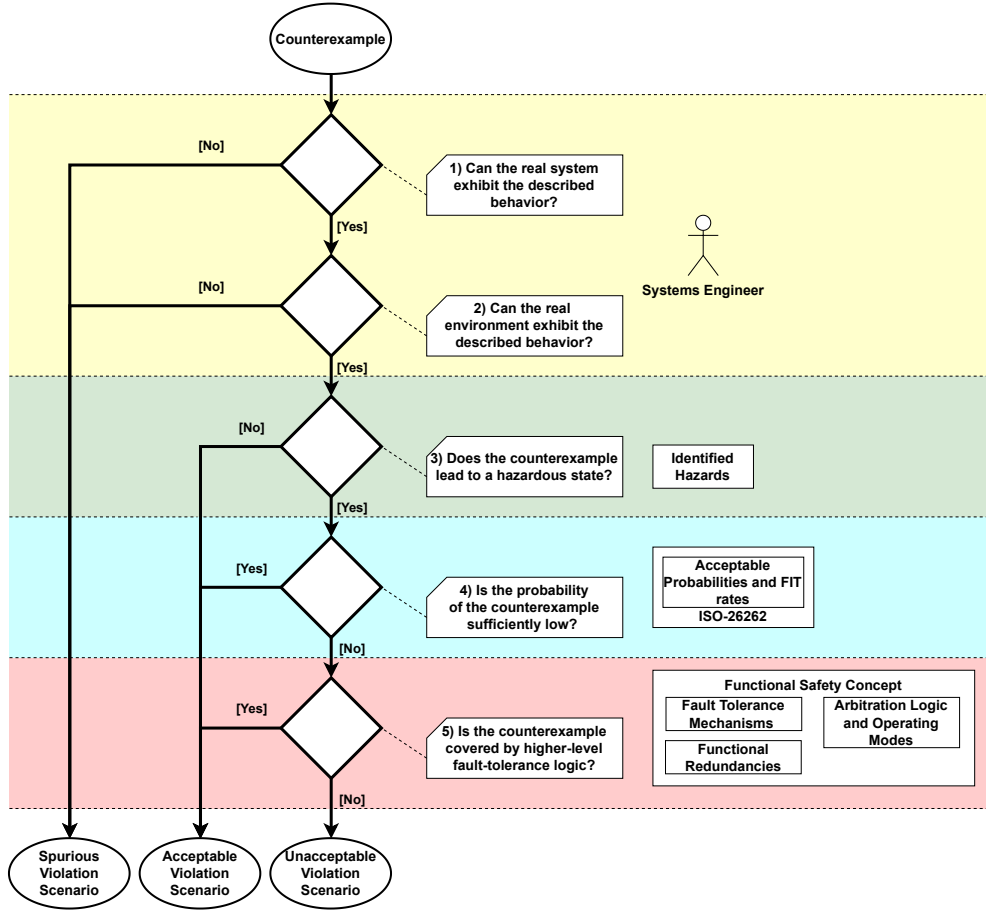


Fig. 6 Categorization Process of the Possible Requirement Violations

- *Irrelevant events*: the previous point can be generalized further, as even if a component is relevant to the exemplified problem, it can still perform irrelevant steps.
- *Loops*: A concrete counterexample specifies a fixed number of loop traversals, while the abstract scenario description can allow for arbitrary repetitions. An example in our case study is the presence of transient failure-repair loops. In theory, this might eliminate an infinite number of traces.
- *Ignoring nominal behavior*: this principle overlaps with the previous ones. As the behavior of reactive systems is typically structured as an infinite loop, a problematic execution can start with an arbitrarily long period of correct nominal behavior before something goes awry. As such, the abstract scenario descriptions should be able to ignore an arbitrarily long prefix of the execution path. In theory, this might eliminate an infinite number of traces if the nominal behavior has loops.

- *Reordering*: the order of some events in an execution path might be irrelevant, especially when they are related to different components. When interaction is present, the whole execution cannot always be arbitrarily reordered, but a part of it might be. For example, if RWA A cannot recover from a transient failure if RWA B is not working, then it does not matter whether RWA A or RWA B fails first, *as long as both happen before RWA A is repaired*. This may eliminate exponentially many traces as a function of the trace length, depending on how many events can be freely reordered.
- *Equivalence intervals (qualitative abstraction)*: if numeric inputs are present in the model, the counterexample will contain one specific number, although several (potentially infinitely many) others can lead to the same behavior. In this case, the abstract scenario description may check for an interval instead of a specific value. The reduction is based on the size of the interval for discrete inputs, while it eliminates theoretically infinitely many possibilities for continuous ones.

More detailed examples are presented in [Section 6](#). This is an open-ended list; the experts might use other principles to base the abstraction on. Although the impact of using abstract scenario descriptions depends on the chosen abstraction, and thus getting infinitely many redundant traces still remains possible in theory, we found that most causes of such redundancy can be eliminated by very simple abstractions like in the list above. If another trace exemplifying the same problem is later returned (so the abstraction was not coarse enough), the expert evaluates why that trace is considered redundant, what difference between that and previous traces does not matter, and then formalizes this in an abstract scenario description.

Note that although the decisions made when analyzing the violation traces are done by humans, and thus may seem subjective, similar decisions are also always made by humans when a formal model is created for the system. The creation of a system’s model always involves human experts with knowledge about the system, as the design must be put into formal mathematical terms before the computer can reason about it. Just like in the classical usage of model checkers, such decisions must be made based on the principles outlined in the given industrial domain’s safety standards and best practices (as can be seen in [Figure 5](#)). Our process aims to make the decisions more granular, thus less error-prone than the creation of a monolithic environment model right at the start and the monolithic formalization of the exact subsystem-level requirements involving the guarantees of higher-level fault tolerance measures. The systematic iterative exploration also helps in documenting the decisions thoroughly, aiding reviews, which is already the primary way for reducing subjectivity when creating the input artifacts for computational analysis.

After this, we go back to *Step 1*. with the modified artifacts.

Ending the iteration

As our proposed process focuses on systematically finding all violation scenarios, the above steps are repeated until no violation is reported in *Step 1*. Unlike in the traditional application of model checkers, the absence of violation at this step does not mean that the component perfectly satisfies the requirement but that the set of abstract

violation descriptions covers all possible violations. The resulting set of known violation descriptions can be used as input for standard reliability analysis methods and higher-level safety controller design. As the categorization of a scenario can be somewhat subjective, a rationale for each decision must be included, and peer review is encouraged.

Output

The output consists of two related artifacts:

- The final formal model, including the environment model formalizing all of the environmental assumptions found throughout the iteration, and the formalized abstract scenario descriptions formalizing the acceptable scenarios, also including the final formalized requirement.
- A table containing the natural language description violation scenarios found throughout the iteration, the categorization, a rationale for the categorization, and the consequence, identifying which parts of the formal model changed as a result of the iteration.

This two-component output is created with two goals:

- As the main goal of the process is to use model checkers as information providers, the tabular format is the main output artifact, compiling all the information about the environmental assumptions and the weaknesses of the design.
- To mitigate subjectivity in the categorization decisions and to validate the consistency of the natural language description and formal description of the abstract scenarios, this tabular format can be easily reviewed.

Guarantees

As long as the abstract scenario description covers the newly found violation, it will not be returned by the model checker again. As the abstract scenario description is created manually, it might not be perfect, leading to two types of possible errors:

1. A violating execution path was accidentally *not covered*: it will appear in another iteration leading to another abstract scenario description, so the final list will still include every important scenario.
2. An execution path was unintentionally *covered*: this is much harder to notice, and we need to consider two sub-cases depending on whether the unintentionally covered trace was violating or not:
 - (a) The unintentionally covered trace is *non-violating*: this is an error on the safe side. It might lead to stricter requirements for the higher-level fault tolerance measures or worse reliability computed.

Mitigation: We can use the model checker with the query $\mathbf{GF}(accept_i) \implies \neg\phi$ (where i th is the new scenario) – if a counterexample for this property exists, then it describes an execution which is covered by the abstract scenario

description, but does not violate the requirement, so covering it was (likely) unintended.

- (b) The unintentionally covered trace is *violating*: This is a more problematic case, as it might cause the natural language descriptions not to cover every weakness, or worse, we might ignore a trace later that we should have deemed unacceptable.

Mitigation: Standard validation techniques for formal models, like expert review and simulation, can be used to check the consistency of the natural language description and its formal representation. As the output of this process includes both the formal abstract scenario descriptions and the natural language descriptions of the scenarios, such validation activities are supported. The scenario automata are much smaller and less complex than the original system, and each of them has to be consistent with only a single natural language description, not a complex specification. As validation activities are needed for the system itself as well, the additional validation of the formal scenario descriptions can be regarded as an acceptable overhead.

Regarding 2b, keep in mind that the main use case of our proposed process is the *exploration* of possible scenarios, working as an information provider, helping systems engineers better understand the less obvious weaknesses of different architectures – the results are not meant to be directly used for a safety case by themselves. The aim is to be better than using only engineering intuition and “mental” or tool-supported simulation to understand the nuances of a design.

We are concerned with only one requirement at a time – otherwise, if an execution E_1 violates requirement R_1 , and the resulting abstract scenario description covers another execution E_2 that violates another requirement R_2 , we would lose the information that R_2 is violated by E_2 , as the model checker will no longer output E_2 at all. This would be a problem if we want to handle R_1 and R_2 independently, in which case the process should be performed independently for them.

4.3 Generalizing the Process to other safety standards

In the scope of this paper, we present how the process can be applied in the automotive domain, building on the ISO 26262 standard. However, the process itself is general and does not depend on the specific application domain or industry; it can be utilized in conjunction with other safety standards as well. Figure 5 and Figure 6 show the relationship between our process and the terms, steps and artifacts used in the system design according to ISO 26262, but there are analogous ones in other standards as well – the main difference we have found in other standards is the lack of a specific *safety concept*, which integrates all safety-related aspects of the system architecture in an early phase, but the necessary information can still be acquired from the form of system documentation prescribed by other standards in safety-critical domains, albeit sometimes in a less concentrated manner. Specifics about how the relations between our process and the safety standards’ steps, terms, and artifacts change in the context of the ECSS (space) and EN 50126-50128 (railway) standards are summarized in

[Table 1](#) and [Table 2](#) to exemplify that the process can be integrated into the design process of other industries as well.

Table 1: Summary of steps, artifacts, and terms of safety standards related to our process

	ISO 26262 (Automotive)		EN 50126–50129 (Railway)		ECSS (Space)	
	Phases and Activities	Artifacts, Documented Information	Phases and Activities	Artifacts, Documented Information	Phases and Activities	Artifacts, Documented Information
Requirements		Safety Goals	Specification of System Requirements (Phase 4)	System requirements RAMS system requirements specification Safety related application conditions	Phase 0 – Mission analysis / Needs identification	Safety Requirements and Dependability Requirements (part of PA-induced requirements)

Continued on next page

Table 1: Summary of steps, artifacts, and terms of safety standards related to our process (Continued)

	ISO 26262 (Automotive)		EN 50126–50129 (Railway)		ECSS (Space)	
	Phases and Activities	Artifacts, Documented Information	Phases and Activities	Artifacts, Documented Information	Phases and Activities	Artifacts, Documented Information
System Architecture	Developing the Item Definition	Preliminary Architecture; Definition of Systems	Architecture and Apportionment of System Requirements (Phase 5)	System architecture (including system interface specifications and system hazard analysis) Allocation of RAMS requirements to components Acceptance criteria Hazard log (updated)	Phase A: Feasibility Trade-off analysis	System Architecture Safety Critical Functions Hazard and risk minimization options
			Design and Implementation (Phase 6)	Subsystems and components RAM analysis Hazard analysis Hazard log (updated)	Phase B – Preliminary definition	Risks and Hazards Design Definition File Fault-tolerant system design

Continued on next page

Table 1: Summary of steps, artifacts, and terms of safety standards related to our process (Continued)

	ISO 26262 (Automotive)		EN 50126–50129 (Railway)		ECSS (Space)	
	Phases and Activities	Artifacts, Documented Information	Phases and Activities	Artifacts, Documented Information	Phases and Activities	Artifacts, Documented Information
Environmental Assumptions	Hazard Analysis and Risk Assessment		System Definition and Operational Context (Phase 2)	System definition: mission profile; boundaries; operational context; scope of risk analysis; RAM plan; safety plan	Phase 0 – Mission analysis / Needs identification	Preliminary risks and hazards Preliminary Technical Requirements Specification
					Phase A – Feasibility	Technical Requirements Specification

Continued on next page

Table 1: Summary of steps, artifacts, and terms of safety standards related to our process (Continued)

	ISO 26262 (Automotive)		EN 50126–50129 (Railway)		ECSS (Space)	
	Phases and Activities	Artifacts, Documented Information	Phases and Activities	Artifacts, Documented Information	Phases and Activities	Artifacts, Documented Information
Known Violation Scenarios	Hazard Analysis and Risk Assessment		Risk Analysis and Evaluation (Phase 3, continuous)	Risk assessment Hazard log Safety plan and RAMS plan (updates)	Phase A: Feasibility Hazard Analysis; Safety Risk Assessment and Control (part of general risk assessment) Dependability Risk Assessment	Hazard and safety risk register Ranked hazard and safety risk log System level hazards Identified safety-critical aspects

Table 2: Summary of artifacts and terms of safety standards supporting the categorization process

	ISO 26262 (Automotive) Artifacts, Documented Information	EN 50126–50129 (Railway) Artifacts, Docu- mented Informa- tion	ECSS (Space) Artifacts, Docu- mented Informa- tion
Can the real system exhibit the described behavior?		System Definition and Operational Context	Design Definition File
Can the real environment exhibit the described behavior?		System Definition and Operational Context	Technical Requirements Specification
Does the counterexample lead to a hazardous state?	Identified hazards	Hazard log	Hazard and safety risk register Ranked hazard and safety risk log
Is the counterexample covered by higher-level fault-tolerance logic?	Functional safety concept (fault tolerance mechanisms, arbitration logic and operating modes, functional redundancies)	RAM plan Safety plan	Design Definition File FDIR documentation
Is the probability of the counterexample sufficiently low?	Acceptable probabilities and FIT rates (ASIL)	Allocation of RAMS requirements to components (THR, TFFR, SIL)	PA-induced requirements Probabilistic safety targets (established by the customer)

4.4 Comparison to Automated Abstraction Refinement Algorithms in Model Checking

The main overall scheme present in our process is reminiscent of the loop often used in model checking algorithms, especially in abstraction refinement: i) a model describing a conservative abstraction of the actual behavior of interest is analyzed first using

some model checking algorithm, ii) if a violation is found, the violating trace is checked to see if it is spurious or not, iii) if spurious, then the model is changed (refined) in a way to eliminate the spurious trace, then it is analyzed again.

This is the backbone of several existing model checking algorithms, where these steps are automated, relying in most cases on abstract domains allowing the automated computation of conservative abstractions for an already existing over-detailed model given as input and the automated derivation of refinements based on violation traces that are present in the conservative abstraction but not in the precise model.

Such methods include the classical *Counterexample-Guided Abstraction Refinement* (CEGAR) loop presented in [Clarke et al. \(2000\)](#) and its lazy variant ([Henzinger et al. 2002](#)), both of which use it in the context of predicate abstraction, relying on SMT solvers capable of producing a proof of infeasibility for spurious violation traces (unsat cores or binary Craig interpolants) to guide the refinement step. Lazy abstraction with interpolants ([McMillan 2006](#)) relies on a similar idea, but uses sequence interpolants to generalize each infeasible trace to rule out other infeasible ones as well after each refinement step (similarly to unsat cores and binary Craig interpolants, but with focusing on the whole trace instead of single cut points).

Trace abstraction ([Heizmann et al. 2013](#)) is the most similar to how our process works: this algorithm constructs automata accepting the language of infeasible program executions (considering program statements as letters and traces as words), and extends these automata after finding infeasible traces in the abstract model – so the automata are used as abstractions of infeasible violation traces, generalizing from a single trace to all of those which have the same reason of infeasibility.

Although there are many similarities on the surface, our process has a significantly different use case and aim, and as such, uses very different solutions to achieve it compared to automated abstraction refinement algorithms. We will first list what is analogous between them to give context to the differences described after that.

- The initial model that our iterative process starts with can be considered a conservative abstraction of a precise model of the system and its environment.
- This current model is analyzed using a model checker – the exact algorithm does not matter, only its capability to produce a violating trace if the property to verify is violated. CEGAR and similar algorithms can also often utilize different “inner” model checking algorithms ([Cimatti et al. 2014](#); [Tonetta 2009](#)) for checking the abstract model, which is similar to how we use a black box model checker.
- As the violation trace might be spurious (only present as a result of abstraction), this must be checked, and then we proceed based on how it was classified. If the trace is spurious, abstraction refinement algorithms change the abstract model to no longer allow this trace – we also do this, as we want a different trace in the next iteration.
- When eliminating a spurious trace, abstraction refinement methods try to generalize it and rule out other traces from the abstract model that have a similar reason for being spurious. Our method uses abstract violation scenarios and their automata representations with a similar goal.

Unlike in the case of automated abstraction refinement algorithms, like CEGAR and its peers, our goal is not to make model checking itself more efficient algorithmically, but to make the usage of model checkers more useful methodologically during system design.

- Our context is different: there is no concrete, (over-)detailed formal model yet, as our process is about the creation of formalized artifacts itself, not the analysis of an existing artifact.
- This makes the exact meaning of *abstract model* different: it is not an automatic conservative abstraction of a concrete formal model, but an abstraction of the system behavior and environmental assumptions whose complete formal description has not been created yet. The different aspects are formalized on demand as the model versions are checked.
- For the same reason, the concept of *spuriousness* is also different: in automated abstraction refinement methods, a trace is spurious when it cannot be performed by the concrete model, which can be checked in an automated way, while in our case deciding spuriousness is based on human expertise (note that this is decided by the same human experts who could have created an over-detailed model in the first place).
- Because of aiming for usability in the context of multi-level fault tolerance and considering random faults, our categorization of violation traces also considers *acceptable* violations, which is not present in automated methods. Giving additional information to the model checker could be used to allow this categorization as well in the CEGAR loop, but then the conditions for acceptability would have to be formalized beforehand, basically already building a model that includes the information of higher-order fault tolerance and probabilities, which is exactly what we found to be impractical and thus try to avoid in our context.
- Generalization from specific traces to abstract violation scenarios is intentionally human-driven in our process, as the goal in our case is the exploration of scenarios. Thus, the person or team whose knowledge about the system's behavior is to be enhanced should decide what aspect of the violation is not important, in what way the rest of the generated violations (if any) must differ from the current one.
- While most of the CEGAR-like algorithms focus only on the abstraction and refinement of the system model, we also consider refining the requirement: when a violation is deemed *acceptable*, that is a refinement of the requirement to be less strict.
- The main output of CEGAR-like algorithms is the binary verdict of whether the model satisfies the property of interest or not – the iteration is only performed in order to find the level of precision necessary to give this verdict. In contrast, our process always iterates until no more violations are found, and the main output is compiled from the results of every iteration.
- While CEGAR and similar algorithms stop when an actual (non-spurious, unacceptable) violation is found, in our process, the architecture is fixed to eliminate this violation, and the iteration continues, as our process is a methodological loop for the design of the system, not for the verification of a single specific model.

- Most implementations of CEGAR are based on exploring the abstract state space while computing the abstract successor relation based on the current precision, and refinement is done by changing this precision. In contrast, we refine the model in less strict ways, either extending the environment model in case of spurious violations, or adding abstract scenario descriptions to the model for the acceptable scenarios – the latter one being similar to how trace abstraction ([Heizmann et al. 2013](#)) eliminates generalizations of infeasible traces.

Some of these aspects (like changing the requirement or changing the model itself when an actual unacceptable violation is found) are, of course, present in the classical iterative way of using model checkers in system design as well, but they are still distinguishing characteristics of our process when compared to CEGAR-like model checking algorithms.

These differences make our process orthogonal to the use case of existing automated techniques. Note that any model checking algorithm can be used in the model checking step of our process to generate a single violation trace, including CEGAR, lazy abstraction, trace abstraction, etc. Conceptually, using an abstraction-refinement-based model checker in our process means that:

- there is an outer, methodological loop that iteratively refines a model that is an abstraction of how the actual (sub)system and environment behave, and a requirement that is an abstraction of what is actually expected from the (sub)system (both of which have not been exactly formalized yet),
- and there is an inner algorithmic loop which verifies the currently formalized aspects by computing an abstraction and refining it in an automated way.

Even considering our process in the framework of CEGAR, and treating it as a (somewhat extended) instantiation of the same underlying scheme, the novelty is still present in how it is applied to the design phase without a whole precise formal model for the system already being present, and how its elements are implemented in the context of fault-tolerant system design.

5 Instantiation of the Process using the Gamma Statechart Composition Framework

In this section, we present an example instantiation of the process using the Gamma Statechart Composition Framework, used to show the realizability of the process. The presented instantiation demonstrates how a generic modeling framework can be utilized in conjunction with the proposed process, and highlighting the key aspects required from the modeling framework to implement the process.

5.1 Gamma Statechart Composition Framework

Several modeling tools have been developed to aid the creation of models with mathematically precise semantics. In our case study, we chose the *Gamma Statechart Composition Framework* ([Molnár et al. 2018](#)), designed to model and verify

component-based reactive systems. It supports the hierarchical composition of components, where the behavior of the lowest-level components is modeled by statecharts. The hierarchical composition enables engineers to divide the system into smaller, reusable subsystems, which can be composed using different compositional semantics to define the system behavior. The framework raises an additional modeling layer over statecharts to describe communication between components, making it suitable for modeling complex, hierarchical systems. Our case study of the ASWA subsystem itself consists of multiple components (sensors, RWAs, FBAs).

Gamma Statecharts

The Gamma Statechart Language (GSL) (Graics et al. 2020) is a configurable statechart language based on a subset of SysML/UML statecharts. In addition to well-known modeling constructs, e.g., ports and interfaces, composite states and orthogonal regions, history states and variables, timeouts to model timed behavior, choice and merge nodes, and fork and join nodes, GSL also contains semantic variation points to customize the behavior of statecharts. Semantic variation points include (1) the evaluation order of guard expressions on transition, (2) conflict resolution order of enabled transitions on hierarchy levels, (3) priorities of enabled transitions on the same hierarchy level, and (4) execution order of orthogonal regions.

Composition semantics

The framework supports multiple composition modes (Graics et al. 2025) defined in the Gamma Composition Language (GCL):

- *Synchronous-reactive*: describes synchronously scheduled components executed in *cycles*. Components communicate via signals. At the beginning of the cycle, each component samples the signals on its inputs and executes its behavior according to them. As sampling is done at the beginning of the cycle, communication between the components is always delayed by one cycle.
- *Asynchronous-reactive*: describes concurrently running components communicating via message queues and reacting to events independently, representing the non-deterministic, concurrent behavior of the system.
- *Cascade*: enables the execution of the components in a predefined order. Similar to the synchronous-reactive mode, components communicate via signals and execute in *cycles* in a specific order. They only sample the signals on their inputs right before execution, enabling a pipeline-like execution of the system. Multiple executions of the same component in a given cycle are also supported.
- *Scheduled asynchronous-reactive*: supports the sequential execution of components. Unlike in the *cascade* mode, components communicate via prioritized message queues.

Formal specification of requirements

The Gamma Property Language (GPL) (Graics et al. 2020) is a property language supporting the definition of CTL* (Emerson and Halpern 1986) properties, which is a superset of computational tree logic (CTL) and linear temporal logic (LTL), combining

path quantifiers and temporal operators. The Gamma Property Language can be used to describe complex formal requirements regarding the behavior of the system.

Traces of system behavior

The Gamma Trace Language (GTL) (Graics et al. 2020) is a high-level trace language for the description of execution traces of reactive systems. GTL supports the description of reached state configurations, variable values, input events coming from the system’s environment, and output events produced by the system.

Low-level formal model

The EXtended Symbolic Transition Systems (XSTS) (Graics et al. 2025) is a low-level modeling formalism for formal verification. XSTS consists of variables that represent the system state and transitions that describe the possible state changes. The framework uses semantic preserving model transformations from GSL to XSTS and from XSTS to the input language of the verification backend.

Table 3: Instantiation of the Process using the Gamma Framework

Artifacts in the Process		Concrete Artifact using The Gamma Framework
Requirements		Textual requirements provided by the engineers.
Known Scenarios	Violation	Abstract violation scenario descriptions in the form of automata defined in GSL.
Environmental Assumptions		Simple assumptions are encoded in GPL property; more complex assumptions are defined in an Environmental Model using GSL.
System Architecture		The behavior of the components of the system are modeled using GSL, and the composition of the components and the communication modes between them are defined using GCL.
Temporal Logic Formulas		GPL is used to describe complex formal requirements regarding the behavior of the system.
Formal Model		For the input language of SPIN, Promela models are generated from the Gamma statecharts using semantic preserving model transformation steps.
Counterexamples		The counterexamples are presented in the trace language of SPIN.
Violation Trace		For the engineers to interpret the requirement violations, the counterexamples are backannotated to GTL to help the engineers interpret the violation traces on the GSL models they defined.

5.2 Instantiation of the Process

The Gamma Framework can formally verify the models using different verification back-ends: it generates a low-level model in the input modeling language of the chosen back-end according to the Gamma semantics, translates the requirement given in the GPL to the property language of the back-end, and lets the model checker handle the verification task. As of the writing of this paper, the supported back-ends (Graics et al. 2020, 2025), are UPPAAL (Bengtsson et al. 1995), Theta (Tóth et al. 2017), SPIN (Holzmann 1997), and NuXMV (Cavada et al. 2014). The low-level counterexample directly given by the model checker would be hard to interpret by the user, so Gamma also transforms the output of the model checker back to the Gamma model and shows a trace at that level. We used the SPIN back-end in our case study, as the nature of our requirements of interest required a back-end with full-fledged LTL analysis capabilities. Table 3 summarizes the instantiation of the process using the Gamma framework and SPIN.

6 Evaluation: Exploring Requirement Violation Scenarios of the ASWA Subsystem

This section presents the formalization of our case study and showcases the application of our exploration process.

6.1 Step 1.: Artifact Preparation

We focus on the *Concept Phase* when the exact behavior of the subsystem components is not necessarily known, but the safety measures and safety mechanisms must be defined in the Functional Safety Concept. Even in this early phase of the system design, functional verification is crucial to show that the envisioned system can satisfy the high-level requirements.

System Architecture

The initial architecture of the ASWA subsystem is based on the functional architecture seen in Figure 3. The sensors connected to the RWA and FBA components and the respective data processing routines differ.

The FBA component can only measure the *relative steering wheel angle* by itself, therefore, it needs a *startup angle* with ASIL D integrity level when initializing. Using the received *startup angle*, it converts the measured relative angle to an absolute angle, then a single FBA component can provide an ASWA with ASIL D integrity.

Each RWA component receives a relative steering wheel angle from two sensors and computes an ASWA with ASIL B integrity. Cross-checking this angle between the two RWAs results in an ASWA with ASIL D integrity, usable as a startup angle by the FBA components. After calculating this startup angle, the RWAs switch to an *incremental* computation mode: in this mode, each RWA can compute an absolute steering wheel angle with ASIL D integrity, as each sensor measurement can be used as a relative angle with ASIL B integrity, then cross-checking two of them results in

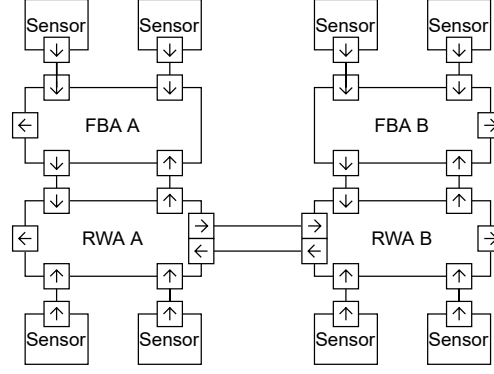


Fig. 7 Architecture of the case study system

a relative angle with ASIL D integrity, and adding this to the startup angle results in an absolute angle with ASIL D integrity.

Each FBA can get a startup angle from the RWA on the same side. There is no communication between the FBAs. The initial architecture can be seen in [Figure 7](#).

Requirements

For the ASWA Measurement subsystem to remain operational and the calculated angle to remain correct, we need both an FBA and an RWA component to be operational to provide sufficiently independent angle calculations.

Environmental Assumptions

Our goal with the use of *environmental assumptions* is to limit the non-deterministic and unrealistic behaviors of the system. Initially, we want to exclude the unrealistic behavior of nothing happening within the system and to limit the number of permanent failures according to the requirement presented in [Subsection 3.3](#).

Known Violation Scenarios

We know our requirement can be trivially violated if both RWAs or both FBAs fail, which is possible given the environmental assumption that two permanent failures can happen in the system. This violation must be handled in the initial formalization step.

6.2 Step 2.: Formalization

The model of our case study subsystem must be modular to support easy modeling of different architectures with different composition semantics, aiding the evaluation of several possible implementations. To enable formal verification, we constructed the formal models using the Gamma Statechart Composition Framework presented in [Subsection 5.1](#).

Functional verification in an early phase can be done using the high-level degradation behavior of the components, which we describe in the *Gamma Statechart Language* (GSL). In the following, we present the high-level degradation model of

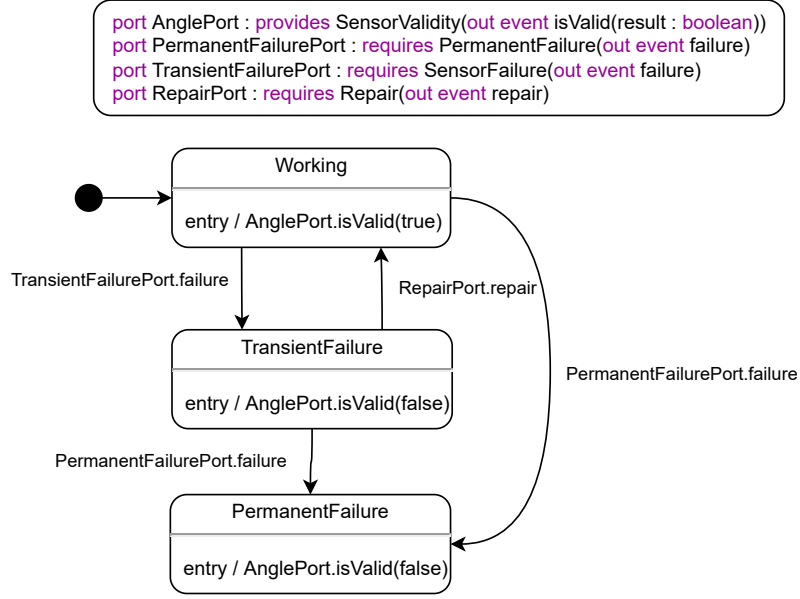


Fig. 8 High-level model of a sensor component

the *sensor*, *FBA*, and *RWA* components. The supplementary material of the paper (Szabó et al. 2023) contains the models and the detailed descriptions of the presented components.

Sensors

From the specifications of the sensors, we know they sometimes send invalid measurements. During the *Concept Phase*, we only consider whether the sensor is providing valid or invalid measurements. Each component might use different kinds of sensors, but their high-level degradation behavior is the same. Therefore, we use an identical state chart for each sensor in the Gamma model.

We defined three states for the sensors: a working state, a state for the case when a transient failure occurred, and a state for the case when a permanent failure occurred.

The sensor provides either valid or invalid measurements (or no measurements, included in the latter to simplify the models). The sensors can fail and get repaired at non-deterministic times, which we model as events coming from the environment, thus, the environment can influence the sensor with failure and repair events. The high-level sensor model can be seen in Figure 8.

Angle measurement in the FBA component

During the FSC phase, the following is known about the component: it needs to be initialized, it reads the measurements from two sensors, it needs a startup angle, its function is to calculate some angle, and it can also experience some errors.

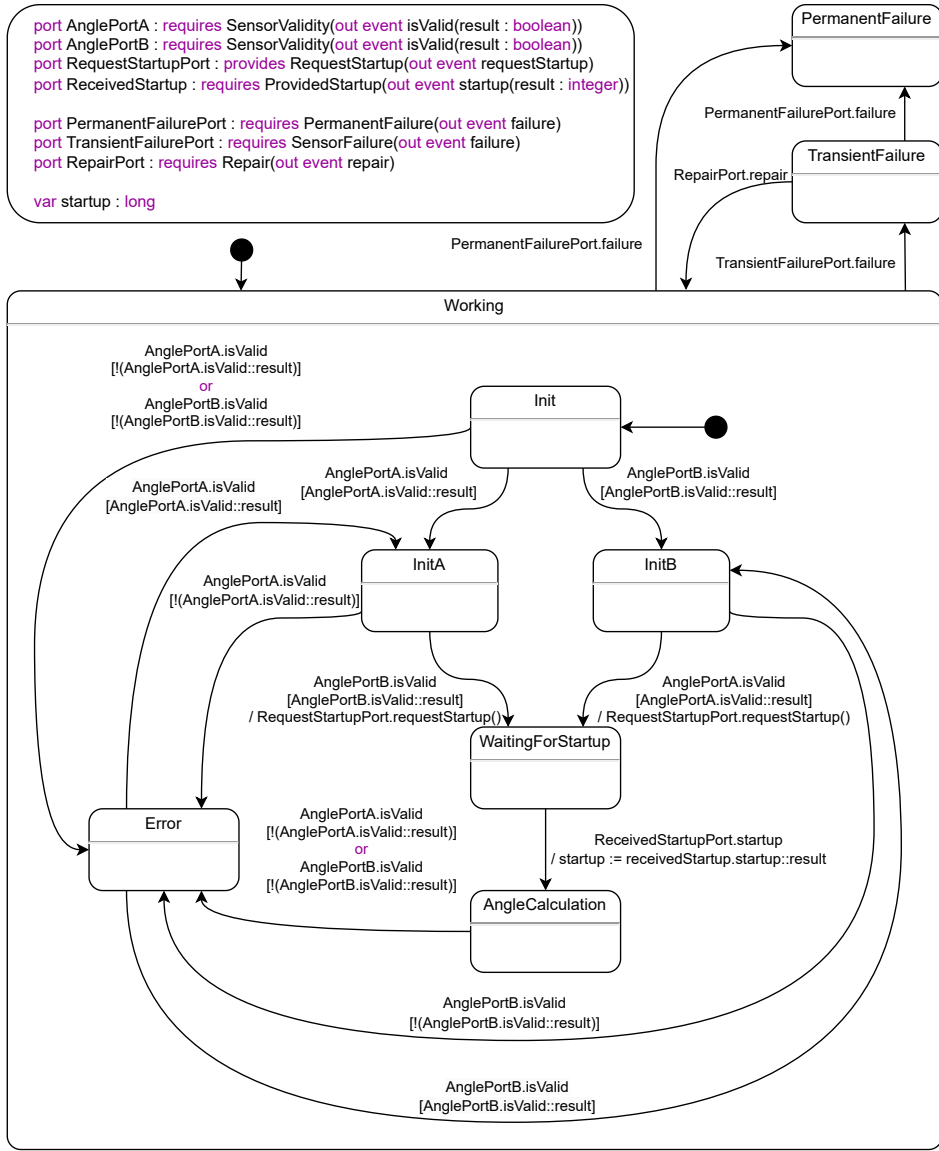


Fig. 9 Model of processing the measurements in an FBA subsystem

The component needs two independent sensor measurements, it must be able to request a startup angle from another component and receive the provided startup angle. The FBA model can be seen in [Figure 9](#).

Angle Measurement in the RWA component

The behavior of the RWA component is similar to that of the FBA component, with an extra cross-checking step between the two sides to compute an ASIL D initial

startup angle and the possibility of providing ASIL B angle after the failure of one of its sensors after the successful cross-check.

The component needs two independent sensor measurements, it can request a startup angle from another component and receive the provided startup angle. The RWA model can be seen in [Figure 10](#).

Composition

We need a composition of the components that can show us the possible propagation of faults and the sequence of faults that can lead to an unsafe system state or the unavailability of the ASWA measurement functionality if such traces exist. Gamma provides various composition semantics for this, presented in [Subsection 5.1](#).

During the *Concept Phase*, the hardware components realizing the system are not necessarily known, preventing the consideration of timing and scheduling constraints. Although the set of possible realizing hardware components can be known given the domain, the scheduling can still be non-deterministic in distributed systems due to jitter, even if the processing units are identical.

The implied information flow from the sensors to the FBA and RWA can somewhat limit nondeterminism in the system.

Given these constraints, we opted for the more permissive *scheduled asynchronous-reactive* composition, which is a good trade-off between allowing non-determinism and enabling engineers to incorporate insights when defining component execution order or to check their assumptions regarding scheduling requirements.

Initial Environment Model

The environmental assumptions are formalized in the *Environment Model*. We introduced three states for the environment for the remaining number of possible permanent failures in the system. Twelve components are connected to the environment model (see [Figure 7](#)), and each of them can get permanent failures, transient failures, and repairs, thus, we introduced ports for each of these. A self-trigger ensures that the environment will not stop providing inputs to the system components. The initial environment model can be seen in [Figure 12](#) and the full composition in [Figure 11](#).

Formalization of the requirement

We formalized our requirement as the following LTL property:

$$\mathbf{GF}((RWA_A.ASIL_D_Calculation \vee RWA_B.ASIL_D_Calculation) \wedge (FBA_A.AngleCalculation \vee FBA_B.AngleCalculation))$$

From the initially known violations, we know that this property can be trivially violated by the permanent failure of both RWAs or both FBAs. According to [decision point 5](#), this is an *acceptable* violation scenario, to be addressed through a higher-level controller that counters it. We modified the property with the assumption that both RWAs or both FBAs cannot fail permanently by extending the original property with an implication. The modified property is the following:

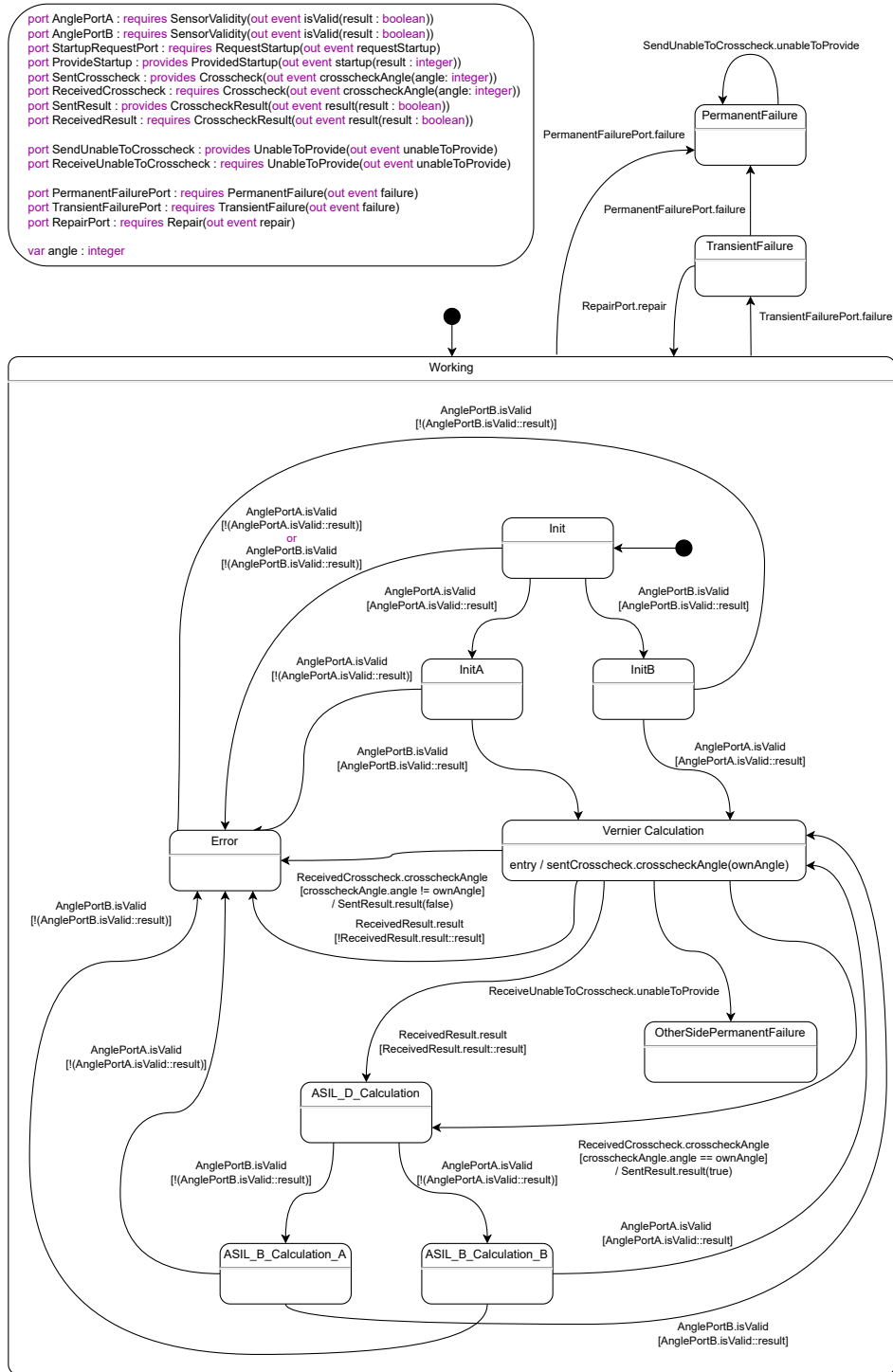


Fig. 10 Model of processing the measurements in an RWA subsystem

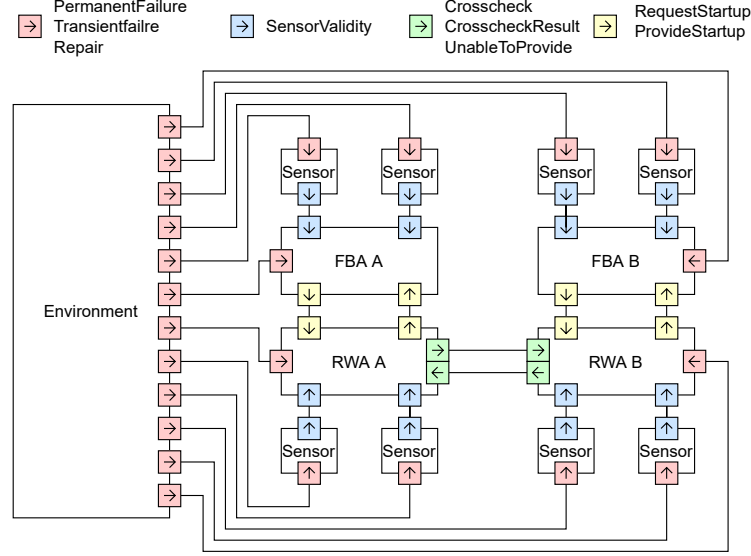


Fig. 11 Composition of the components and the environment model

$$\begin{aligned}
& \mathbf{GF}(\neg(RWA_A.PermanentFailure \wedge RWA_B.PermanentFailure) \wedge \\
& \neg(FBA_A.PermanentFailure \wedge FBA_B.PermanentFailure)) \implies \\
& \mathbf{GF}((RWA_A.ASIL_D_Calculation \vee RWA_B.ASIL_D_Calculation) \wedge \\
& (FBA_A.AngleCalculation \vee FBA_B.AngleCalculation))
\end{aligned}$$

By the semantics of implication, this property can be violated only if the left-hand side of the implication is true, so our assumption holds, and the right-hand side is false, so the original property is violated. This is exactly what we are looking for.

6.3 Step 3.: Violation Analyses

Refining the Environmental Assumptions

The environmental assumptions are captured in the environment model, which we need to modify after running the model checker. The initial environment model ruled out *spurious* behavior when the system does not get any event from the environment, but it enabled sending failure signals to the same component over and over, which is also a *spurious* behavior, as a component cannot fail twice (see [decision point 2](#)). The trace of this behavior can be seen in [Figure 13](#) in high-level form.

We ruled this out adding the condition, that only working components can fail, and only transiently failed components can be repaired.

The first update of the environment model showed us a violation when a component is infinitely kept in a fail-repair cycle. The trace of this behavior can be seen in [Figure 14](#).

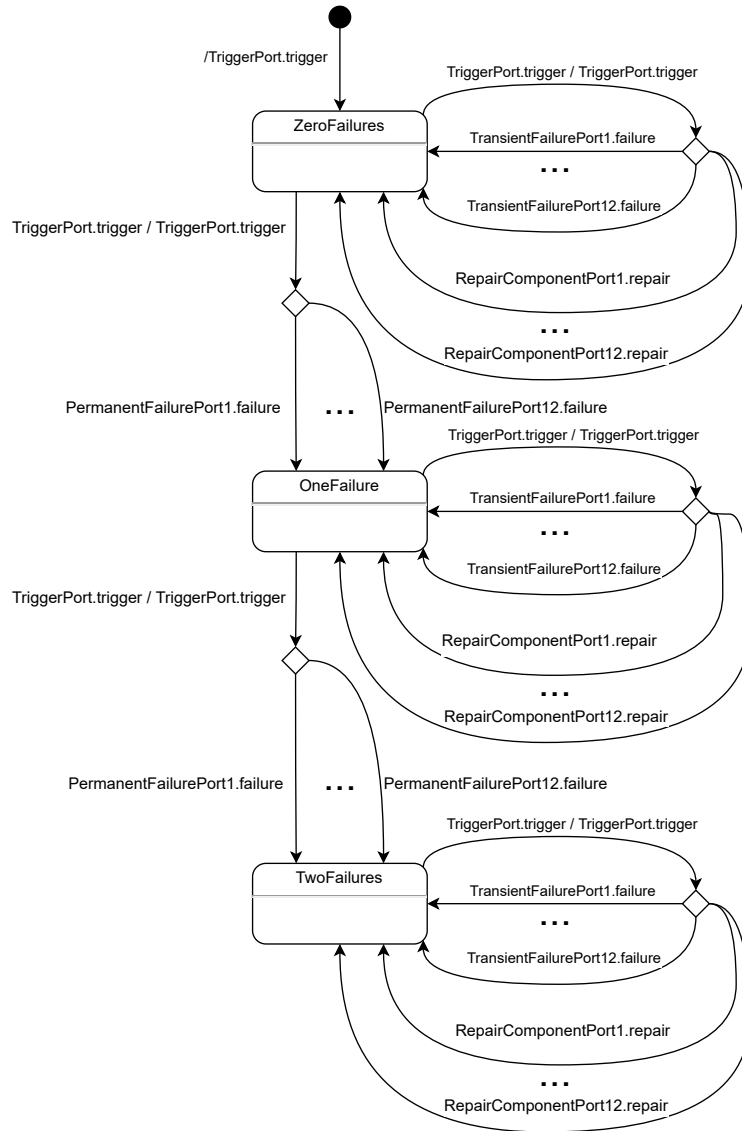
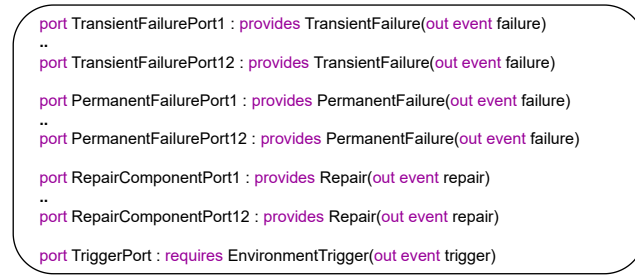


Fig. 12 Model of the initial environment

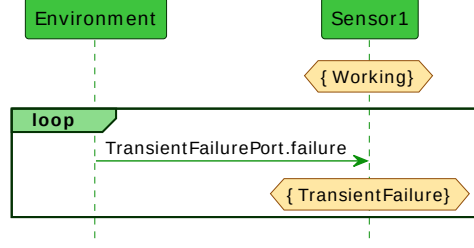


Fig. 13 Trace of a component failing continuously

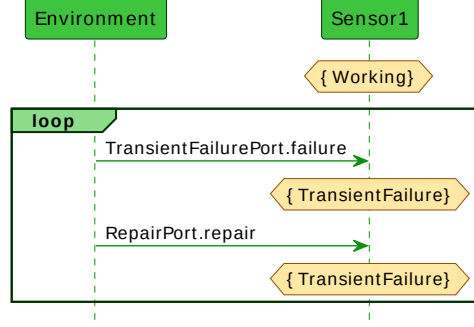


Fig. 14 Trace of a component in a fail-repair cycle

According to [decision point 2](#), this is also a *spurious* behavior as the number of transient failures that can happen in the system must be finite, also, it is an unfair scheduling of the system that only a single component receives events from the environment. After consulting with the safety engineers we found that for the *Concept Phase* it is sufficient to limit the possible transient failures to a fixed number, thus we ruled out this behavior by updating the environment model with the condition, that only a fixed finite number of transient failures can happen in the system.

Modifying the System Architecture

The second update of the environment model showed us that permanent failure of RWA_A or RWA_B and transient failure of the other RWA can violate the requirement, as the RWA cannot re-initialize without cross-checking with the other one. One permanent failure and one transient failure can lead to a violation, based on [decision point 5](#), this was evaluated as an *unacceptable* scenario.

To counter this, the engineers modified the architecture. If the FBA has been properly initialized, we can consider it sufficiently independent to cross-check with; thus, the engineers added the possibility to cross-check between the RWA and the FBA of the same side.

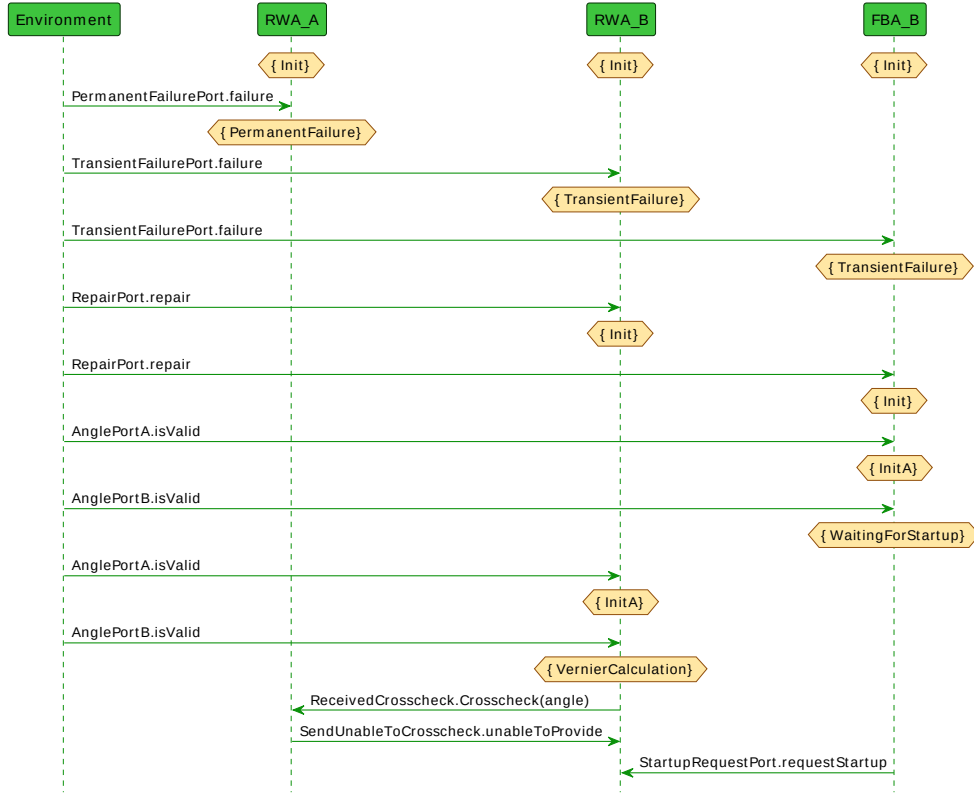


Fig. 15 Trace of components transiently failing before the other can re-initialize, no crosscheck can happen

Handling Acceptable Violation Scenarios

Using the modified architecture, we found that one permanent failure and two transient failures close enough in time to each other can still violate the requirement: they can make the FBA and RWA mutually wait for each other.

First, RWA_A or RWA_B permanently fails, leaving the option for the remaining RWA to cross-check with the same side FBA. If the remaining RWA and the same side FBA transiently fail, they cannot re-initialize properly, as the FBA needs a startup angle from the RWA, which cannot be provided since the RWA is not yet re-initialized, and vice-versa. The trace of this behavior can be seen in [Figure 15](#).

This violation is *not spurious*, but its probability was deemed sufficiently low by the experts, which makes it *acceptable* according to [decision point 4](#). From the trace, we derived an *abstract violation scenario automaton* seen on [Figure 16](#). We defined four states, one of which is *accepting*.

The automaton accepts the traces where after the permanent failure of RWA_A, either FBA_B fails transiently and sometime after that RWA_B fails transiently without the successful re-initialization of FBA_B, or vice versa.

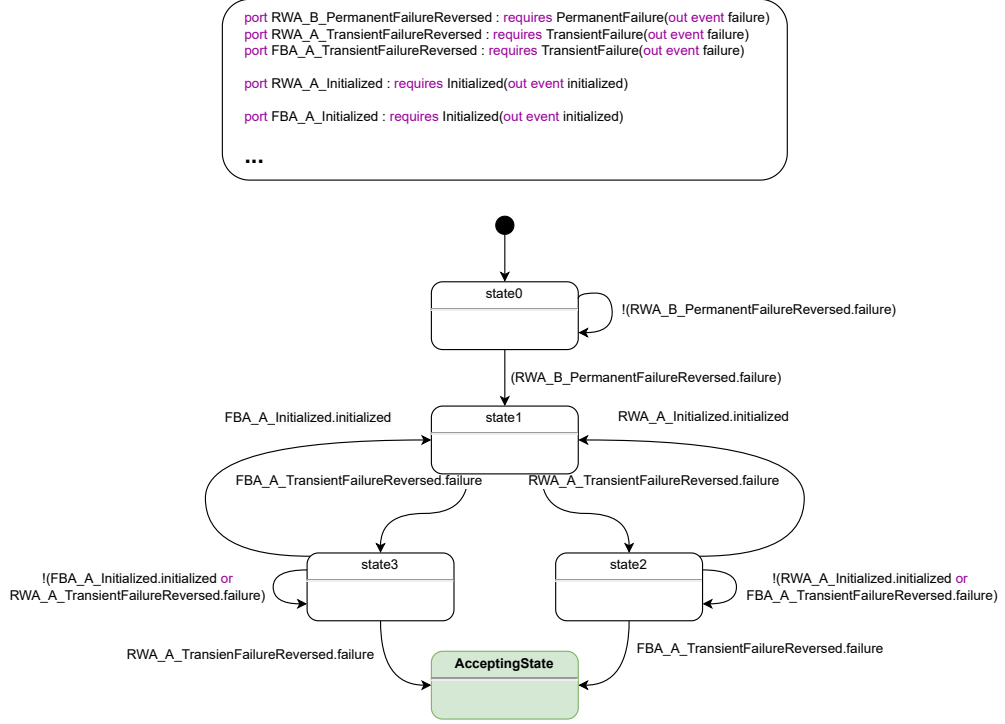


Fig. 16 Model of the abstract violation scenario

In *state0*, *state1*, *state2* and *state3* we abstracted events *irrelevant* to the given state. In *state0*, we are waiting for the event when the RWA_A permanently fails, other events can happen arbitrarily on the trace, thus here we are *ignoring the nominal behavior* of the system. In *state1*, we are waiting for the transient failure of FBA_B or the transient failure of RWA_B, here, all other events are also *irrelevant*. The trace showed us the ordering of events when first the FBA_B transiently fails, then RWA_B transiently fails, but from the system design, we know that these events could be reordered such that the RWA_B fails first, then the FBA_B transiently fails, thus *state2* and *state3* captures this possible reordering of the events. In *state2*, an arbitrary number of events can happen before either the FBA_B transiently fails, which causes the change to the *accepting state*, or the RWA_A is successfully reinitialized, and the nominal behavior can continue. Here we are *ignoring nominal behavior* (e.g, getting sensor information, other components being repaired, etc.), and this behavior can continue for arbitrarily long repetitions and reordering of events. A similar argument could be made about *state3*.

By using the symmetry of the system, we also defined another scenario automaton for the trace where RWA_B fails permanently and FBA_A and RWA_A must fail transiently, similar to the one presented.

We composed the system and the environment model with the two scenario automata, broadcasting the output events of the system and the environment to the corresponding input ports of the scenario automata. In each step, we first execute the system and then the two scenario automata.

We extended the property with the criterion that none of the scenario automata can accept the violation trace. By broadcasting the output events of the system and the environment model to the input ports of the scenario automata, this solution can be theoretically extended to any number of *abstract violation scenario automata*. The extended LTL property, where *Monitor1* and *Monitor2* are the two scenario automata, is the following:

$$\begin{aligned} & \mathbf{GF}(\text{Monitor1}.AcceptingState) \vee \mathbf{GF}(\text{Monitor2}.AcceptingState) \vee \\ & \left(\mathbf{GF}(\neg(RWA_A.PermanentFailure \wedge RWA_B.PermanentFailure) \wedge \right. \\ & \quad \neg(FBA_A.PermanentFailure \wedge FBA_B.PermanentFailure)) \implies \\ & \quad \mathbf{GF}((RWA_A.ASIL_D.Calculation \vee RWA_B.ASIL_D.Calculation) \wedge \\ & \quad \left. (FBA_A.AngleCalculation \vee FBA_B.AngleCalculation)) \right) \end{aligned}$$

After applying the scenario automata, we found that even if transient failures cannot happen close to each other in time, the property can still be violated if the components cannot initialize successfully first. According to [decision point 3](#), this is an *acceptable* violation scenario: if the components cannot initialize at the ignition of the vehicle, the vehicle cannot start, which is a safe scenario. The trace of this behavior can be seen in [Figure 17](#)

We ruled this out by modifying the LTL property with the assumption that all components must successfully initialize:

$$\begin{aligned} & \mathbf{GF}(\text{Monitor1}.AcceptingState) \vee \mathbf{GF}(\text{Monitor2}.AcceptingState) \vee \\ & \left[\left(\mathbf{GF}(\neg(RWA_A.PermanentFailure \wedge RWA_B.PermanentFailure) \wedge \right. \right. \\ & \quad \neg(FBA_A.PermanentFailure \wedge FBA_B.PermanentFailure)) \wedge \\ & \quad \mathbf{F}(RWA_A.initialized) \wedge \mathbf{F}(RWA_B.initialized) \wedge \\ & \quad \left. \mathbf{F}(FBA_A.initialized) \wedge \mathbf{F}(FBA_B.initialized) \right) \implies \\ & \quad \mathbf{GF}((RWA_A.ASIL_D.Calculation \vee RWA_B.ASIL_D.Calculation) \wedge \\ & \quad \left. (FBA_A.AngleCalculation \vee FBA_B.AngleCalculation)) \right] \end{aligned}$$

This modification showed us that the permanent failure of RWA_A, the permanent failure of FBA_B, and the transient failure of either the FBA_A or the RWA_B can violate the property, which is an *unacceptable* scenario (see [decision point 5](#)), and led to the conclusion that the architecture should be modified with additional communication

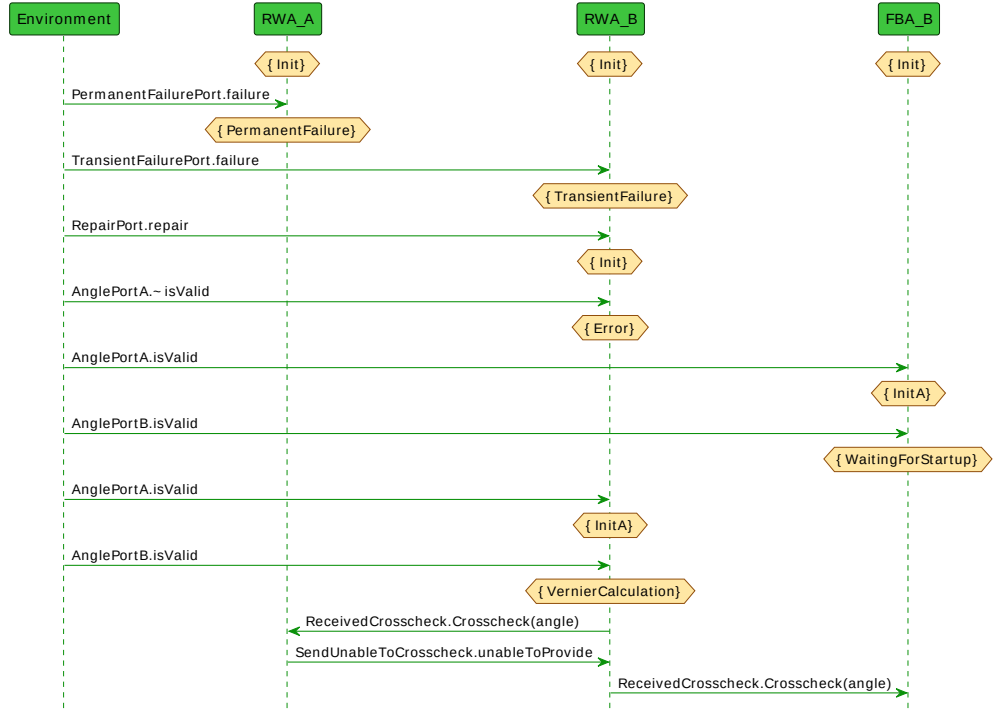


Fig. 17 Trace of the behavior when components cannot initialize properly, halting the crosschecking process

channels. For this modification, the engineers had to negotiate with the OEM, and the output of these negotiations is out of scope for our work. After the negotiations, the process could be continued with the resulting decisions.

7 Results and Discussion of Applying the Exploration Process to the ASWA Subsystem

7.1 Results

We applied the steps discussed in [Section 4](#) to an industrial system currently under development, with a prototype already implemented for the concept. Some violations found by the systematic exploration had already been known to the designers before applying our method, but finding them through reviews took a lot of time, which can be reduced by incorporating the proposed method. The application of the proposed violation scenario exploration process proved the previously known property violations when two permanent failures could make the system fail, but it also showed limitations previously unknown to the system designers, like the scenario of two transient failures happening close to each other in time after a single permanent failure. The probability

of this scenario is low only if there is *no common cause transient failure* for those components, which must be kept in mind for the implementation.

With the iterative application of the exploration process, we have investigated the effect of additional communication channels, enabling more informed decisions about the trade-off between cost and reliability. The architecture of the system can be improved based on the violations found and the collected violation scenarios can be used for further quantitative safety analysis.

Table 4 summarizes the requirement violation scenarios found in our case study.

Table 4: Summary of all violation scenarios found

Scenario	Type	Rationale	Consequence
#1: Both FBAs fail	acceptable	This behavior will be addressed through a higher-level controller that counters it (see decision point 5).	<i>Modification of the property</i> – Degradation of driving experience, but the vehicle can still be controlled.
#2: Both RWAs fail	acceptable	This behavior will be addressed through a higher-level controller that counters it (see decision point 5).	<i>Modification of the property</i> – The vehicle is controlled through an alternative steering function.
#3: No events happening in the system	spurious	After the successful ignition of the vehicle the system must receive events from its environment (see decision point 2).	<i>Refinement of the assumptions</i> – Environment model is forced to send an event in every step.
#4: The environment model constantly sends a failure signal to already failed components, transient failure never gets repaired	spurious	Only working components can fail and only failed systems can be repaired (see decision point 2).	<i>Refinement of the assumptions</i> – The Environment model is constrained to send failure signals only to working components and repair signals only to transiently failed components.

Continued on next page

Table 4: Summary of all violation scenarios found (Continued)

Scenario	Type	Rationale	Consequence
#5: The environment model constantly sends a failure signal followed by a repair signal to the same component over and over.	spurious	Infinite transient failures can simulate permanent failures, thus the number of transient failures the system can experience must be constrained. In the <i>Concept Phase</i> it is sufficient to limit the possible transient failures to a fixed number (see decision point 2).	<i>Refinement of the assumptions</i> – The number of transient failures is constrained in the environment model to be finite.
#6: The environment sends inputs to only one of the components for infinite steps (starvation of other components)	spurious	In a distributed system it cannot happen that only one component receives events from its environment, and all sensors and ECUs work with approximately the same frequency (see decision point 2).	<i>Refinement of the assumptions</i> – Environment model is constrained to send another event to a component only after all the other components have already received something.
#7: After a permanent failure of RWA_B, RWA_A cannot return to ASIL D after a transient error of any of its sensors	unacceptable	One permanent failure and one transient failure can lead to a requirement violation. If the FBA has been properly initialized we can consider it as sufficiently independent to cross-check with (see decision point 5).	<i>Modification of the architecture</i> – Additional communication channels added and the FBA can also provide the startup angle if it has been initialized earlier.

Continued on next page

Table 4: Summary of all violation scenarios found (Continued)

Scenario	Type	Rationale	Consequence
#8: After the permanent failure of RWA_A if both RWA_B and FBA_B experience a transient failure before the first one can reinitialize using the information from the other, then neither can reinitialize	acceptable	As long as there are no common-cause errors in the system it has very low probability, that the same-side RWA and FBA are experiencing transient failures at the same time (see decision point 4).	<i>Extension of known violation scenarios</i> – Abstract violation scenario automaton defined, to constrain the behaviors when the same side components cannot re-initialize before another transient failure happens.
#9: If all the components can't properly initialize first, transient failures can bring the system to a state where the components cannot re-initialize successfully	acceptable	If the system cannot initialize successfully at the ignition of the vehicle, the vehicle cannot start, which is safe (see decision point 3).	<i>Modification of the property</i> – Requiring the successful initialization of the components.
#10: After a permanent failure of RWA_B and FBA_A, RWA_A and FBA_B cannot re-initialize after a transient error of any of their sensors	unacceptable	The architecture must be modified for the system to fulfill the safety requirement (see decision point 5).	<i>Modification of the architecture</i> – Additional communication channels have been added; any component can provide cross-checking or startup angle if they are in a properly working state.

7.2 Comparison to Naive Generation and Other Methodologies

Comparison of abstract violation scenarios and naive trace generation

The SPIN model checker is capable of providing more than one counterexample during one run.¹ To evaluate the applicability of *abstract violation scenarios* descriptions, we generated 200 different counterexamples on the model resulting after the modification of counterexample #7 “naively” (i.e., simply asking the model checker to return 200 violation traces), using the following command: `spin -search -n -e -c200 -m10000 -w27 -DVECTORSZ=4096 -DNOBOUNDCHECK`.

The *abstract violation scenario* automaton seen on [Figure 16](#) covered the first 158 counterexamples before the first counterexample related to counterexample #9 was found. This means that the engineer using SPIN naively would have had to

¹See runtime option -e and -cN on <https://spinroot.com/spin/Man/Pan.html>

examine 158 counterexamples, which have very low probability, before finding the counterexample requiring the modification of the formal property.

Figure 18 shows the increase in the state space of the SPIN model checker. As the initial model of the system is refined through the modifications required by the violation scenarios, the number of explored states increases (SPIN outputs only the number of explored states, not the whole state space of the model, as it stops exploring after the expected number of violations are found). In our use case, this is the expected behavior since each modification results in a more detailed system model and more realistic environmental assumptions, resulting in a bigger state space which must be searched in order to find additional requirement violations. The size increase can come both from introducing new state variables when refining the environment model and from the need to explore additional parts of the state space when looking for new violations not covered by the scenario automata.

The increasing state space size shows an additional advantage of the use of *abstract scenario automata*: in the case of violation scenario #9, using the *abstract scenario automata* halves the searched state space, and enables the verification of more complex system models and finding additional distinct violation scenarios, e.g. #10. In addition to the stored state space, Table 5 also shows the number of explored transitions in the state space, which the use of *abstract scenario automata* reduces even more.

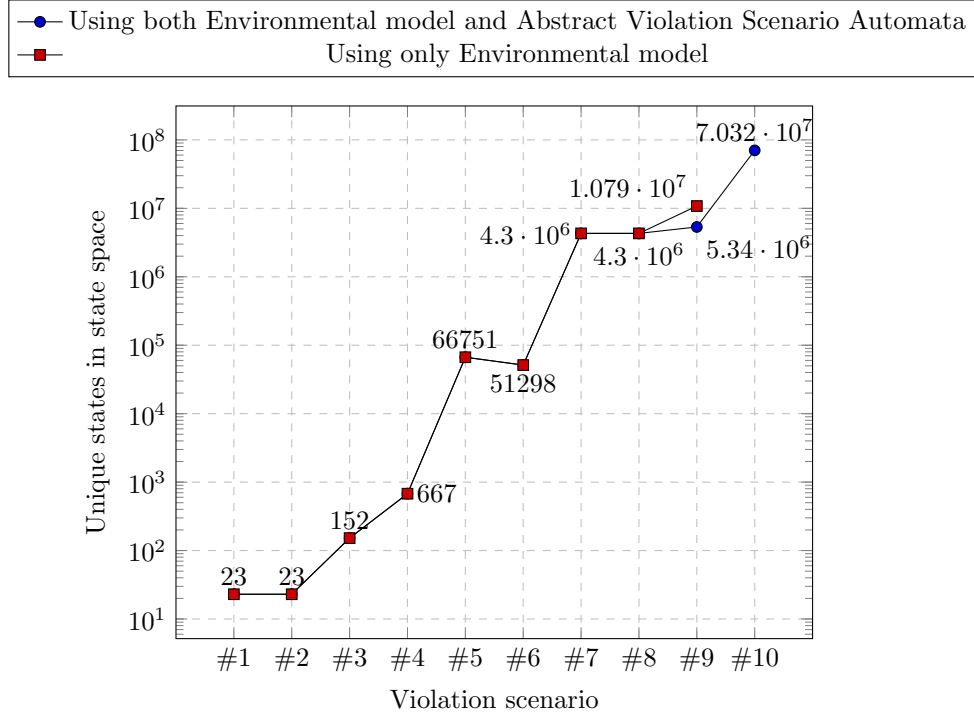


Fig. 18 States stored in state space per violation scenario

Table 5: Quantitative results

Scenario	Unique global system states stored in the state space	Transitions explored during the search
#1	23	32
#2	23	48
#3	152	1243
#4	677	30535
#5	66751	454183
#6	51298	260801
#7	$4.3 \cdot 10^6$	$1.584 \cdot 10^7$
#8	$4.3 \cdot 10^6$	$1.584 \cdot 10^7$
#9	$1.079 \cdot 10^7$ ^a / $5.34 \cdot 10^6$ ^b	$7.762 \cdot 10^7$ ^a / $1.849 \cdot 10^7$ ^b
#10	$7.032 \cdot 10^7$ ^b	$3.158 \cdot 10^9$ ^b

^a Using only Environmental model.

^b Using both Environment model and Abstract Violation Scenario Automata.

Comparison to other methodologies

The industry standard iterative model checking methodology was introduced in (Baier and Katoen 2008). The authors categorize the sources of counterexamples as (1) actual errors in the system, (2) errors made while modeling the system, (3) inaccurate natural language requirements, (4) inaccurate temporal logic formulae, and (5) over-simplification of the model. Our classification is similar. *Spurious counterexamples* cover (2), (3), and (4). However, the standard iterative model checking methodology uses an unconstrained environment; this means that counterexamples #3, #4, #5, and #6 must be encoded in the temporal logic formula, which can be difficult for engineers without expert knowledge. Using statemachines comes more natural to engineers and makes environmental assumptions more explicit. *Unacceptable counterexamples*, e.g., counterexamples #7 and #10, cover (1), (5) and similarly to their methodology, require modification of the system or the formal model. *Acceptable counterexamples*, e.g., counterexample #8, are not covered by their methodology and can only be handled by modifying the formal models and logic formulas. The goal of the *Concept Phase* is to analyze the weaknesses of the different concepts; continuous modification of the formal models makes this analysis harder.

As we can see, other iterative model checking methodologies can also find the design issues of the system, but the presented methodology can be more efficient in cases where quick iteration over the different design options are needed, or the system can produce several near identical violation traces.

7.3 Discussion

Industrial feedback

We worked together with the system designers during the development of this case study, and the categorization of each trace was based on their judgment. Our case study has shown the merits of incorporating our method into industrial processes, as it was able to highlight hard-to-find limitations and potential design flaws in an automotive subsystem currently under development. The iterative exploration of the requirement violation scenarios ensured the completeness of the analysis in the context of the architecture model with the given level of detail, which would have been hard to ensure in an analysis performed solely by the engineers. The presented process provided great help in the evaluation of different development directions and concepts and assisted in evaluating trade-offs between reliability and cost (e.g. the cost of including additional communication channels in the original design).

Another advantage of the systematic exploration process is that assumptions about realistic traces are made explicit through the resulting environment model and the rationales for categorizing a trace as spurious, and the unacceptable violations provided an exact list of weak points of the design, which had to be covered by safety mechanisms in order to improve the system.

The presented process could also be used to determine which development idea is worth allocating resources to more detailed elaboration and further investigation. The use of formal verification provided a strong basis for the engineers in the negotiations with OEMs by showing the advantages and possible disadvantages of different solutions.

The final results of the process revealed that the availability of the ASWA subsystem, along with the availability of the steering in the vehicle, would be greatly improved if the communication between the components of the subsystem did not take place exclusively on the internal, private communication channels, but as an alternative communication route, vehicle communication network would also be used by the components of the subsystem. Since the vehicle communication network is not a resource the steering system exclusively owns, the involvement of the OEMs was necessary to make the solution feasible in practice. The solution used an already established connection, so it did not require significant investment from the OEM side, while it provided significant advantages in the availability of steering functionality in the vehicle. Based on these, the engineers in agreement with the OEM, decided to develop the solution in detail and prepare it for introduction.

Limitations of the presented process

A possible limitation of the process is the creation of abstract scenarios from the concrete violation traces, which still requires manual work and extensive knowledge of the system from the engineers.

Another limitation is the potential difficulty of creating appropriate abstract scenario descriptions. In our case study, we used state machines for this, but they can be replaced by other, more practical description languages, like message sequence charts (Harel and Thiagarajan 2003). This is a challenge in the *implementation* of the process, not a flaw of the underlying ideas.

8 Related Work

Methodology

Several works in the literature present methodologies for using model checking during the development of safety-critical systems (Lahtinen et al. 2012; Kang et al. 2013; Stewart et al. 2021; Mader et al. 2013; Kaleeswaran et al. 2023; Ni et al. 2015). Our process can be integrated with most of these, as they either present systematic or automated approaches for creating formal models in a specific domain or based on a specific artifact (Lahtinen et al. 2012; Kang et al. 2013; Stewart et al. 2021; Mader et al. 2013; Ni et al. 2015), or extend the classic iterative approach in ways orthogonal to ours (Kaleeswaran et al. 2023).

Rushby (2002) used model checking to uncover differences between human “mental models” of how an autopilot should behave and the actual logic of an autopilot. Here, model checking was used as a way to provide information to human factor experts and to explore the consequences of alternative designs, not to fix the logic, which is similar to our goal.

Several methods have been proposed for automatically generating reliability analysis artifacts (fault trees, FMEA tables, etc.) from system models (Bittner et al. 2016a; Kölbl and Leue 2018; Bittner et al. 2016b; Elmqvist and Nadjm-Tehrani 2008; Mader et al. 2013). Incorporating a human expert in our process can be considered a disadvantage compared to the fully automatic approach of xSAP (Bittner et al. 2016a), but it also encourages the system designers to be more aware of the requirement violation scenarios. The OASIS tool (Mader et al. 2013) needs manually defined error propagation models to synthesize the system-level artifacts. Our exploration process can help with making sure that all possible erroneous behaviors are taken into account when defining such error models. The algorithm used for synthesizing safety interfaces by (Elmqvist and Nadjm-Tehrani 2008) is constrained to safety properties, it cannot handle liveness.

The xSAP tool (Bittner et al. 2016a) can be used to extend nominal behavior models of system components with failure models, and then automatically compute cut sets using an efficient algorithm tailored to this task (Bozzano et al. 2015b) implemented in nuXmv. Our method does not assume any specific algorithm in the model checker, and the inclusion of an expert makes it possible to vary the level of abstraction dynamically for each violation scenario. While xSAP can generate FTs with Priority AND gates to encode ordering constraints, it still cannot handle transient failures well, while our method can.

A data-driven fault tree construction algorithm has also been proposed (Jimenez-Roa et al. 2023), which is usable in the operation phase when failure data is available, unlike our approach which focuses on the design phase.

Using model checkers to validate the assumptions used while manually constructing fault trees is described by Cha et al. (2003), which can be considered the “inverse” of one of our use cases. Ortmeier et al. (2003) presents a method for integrating the results of model checking and fault tree analysis after intentionally developing them independently.

Although model checking is often performed using unconstrained environment models to err on the safe side (Lahtinen et al. 2012), others have also noted that such verification can often lead to too many spurious violations (Giannakopoulou et al. 2002). Automated ways have been proposed to infer the environmental assumptions of a system (Giannakopoulou et al. 2002; Gaaloul et al. 2022), synthesizing the constraints for the environment under which the system satisfies its requirements. Such methods could be used instead of manually formalizing the assumptions. However, the automatically synthesized constraints might be hard to interpret and thus hard to review (which is necessary for safety assessment) and to validate whether they are consistent with the system designer’s implicit assumptions. Another problem is that such synthesis automatically gives an environment that makes the system satisfy the requirement, it would also mask all non-spurious violations.

The validation process described by Gouyon et al. (2020) allocates the responsibility of describing constraints for each operational scenario to human experts, similar to how we handle creating the abstract scenarios from the concrete violation traces.

Industrial studies using complex requirements

Industrial studies for model checking often focus on reachability properties, as the most common requirement is avoiding an erroneous or unsafe state. Some works using more complex requirements formalized in LTL, CTL, or some other temporal logic, similar to ours, are also available (Pakonen et al. 2021; Lahtinen et al. 2012; Bernardeschi et al. 2001; Bozzano et al. 2015a; Bernardeschi et al. 2002; Miller et al. 2005).

Studies of automotive subsystems

Several safety- and reliability-focused industrial studies are available from the automotive domain (Huang and Li 2020; Sharvia and Papadopoulos 2015; Ghadhab et al. 2019; Kang et al. 2013; Kölbl and Leue 2018; Bozzano et al. 2020; Huang et al. 2019; Rupanov et al. 2014; Munir and Koushanfar 2020; Poudel and Munir 2021). Our work adds to the diversity of publicly available automotive case studies: to our knowledge, there is no publicly available case study regarding the safety verification of steer-by-wire systems at a similar level as ours (the ones analyzing SbW systems are mostly concerned with higher-level architectural decisions (Huang and Li 2020; Poudel and Munir 2021)).

9 Conclusion

We proposed a novel process for exploring requirement violation scenarios in subsystems of fault-tolerant systems using model checking tools. Unlike how such tools are commonly used in the assessment of safety-critical software systems, we want to explore all of the possible ways the requirements can be violated instead of designing a system that is perfect w.r.t. a given qualitative requirement.

In our process, an expert checks the requirement violation scenarios and decides whether **a)** the violation trace shall be handled in the analyzed subsystem by changing the design, or **b)** the formal models of the system or the environmental assumptions must be refined to describe the system better, or **c)** the violation can be deemed as

a trace of low probability that is allowed according to the safety integrity level, or the violation is to be handled by fault tolerance implemented at higher level of the system hierarchy. In the latter case, the abstract representations of the violation traces are used in the iterative model checking process to filter out the generation of other violation scenarios similar to the traces the expert already accepted.

We presented the applicability of our approach in an automotive industrial study. As a result of the process, **a)** assumptions about realistic behaviors have been made explicit, **b)** the system design has been improved to avoid unacceptable violations found in the original architecture, and **c)** a list of “acceptable” violation scenarios has been compiled usable for designing higher-level fault tolerance measures and performing reliability analysis.

We use the model checker as a black box, so any other model checking tool can be used if it provides a counterexample trace. Our study used the Gamma Statechart Composition Framework, but the process is also agnostic to the modeling language and tool used.

The applicability of the process is not constrained to the automotive industry either; it can be incorporated into the design phase of other safety-critical systems and can be tailored to fit into methodologies created with other safety standards in mind.

Declarations

- Funding

Financial support was received from the following grants:

- Project no. 2019-1.3.1-KK-2019-00004 has been implemented with the support provided from the National Research, Development and Innovation Fund of Hungary, financed under the 2019-1.3.1-KK funding scheme.
- The project supported by the Doctoral Excellence Fellowship Programme (DCEP) is funded by the National Research Development and Innovation Fund of the Ministry of Culture and Innovation and the Budapest University of Technology and Economics, under a grant agreement with the National Research, Development and Innovation Office.
- This research was partially funded by the 2024-2.1.1-EKÖP-2024-00003 University Research Scholarship Programme under project number EKÖP-24-3-BME-159 with the support provided by the Ministry of Culture and Innovation of Hungary from the NRDI Fund.

- Conflict of interest

BME has a research and consultation contract with thyssenkrupp Components Technology Hungary Ltd.

- Materials availability

The models presented in this paper are available as supplementary material: <https://doi.org/10.5281/zenodo.10256749>.

- Author contribution

CRediT authorship contribution statement:

Richárd Szabó: Conceptualization, Methodology, Software, Writing - Original Draft. **Dániel Szekeres:** Conceptualization, Methodology, Writing - Original

Draft. **Simon József Nagy:** Writing - Original Draft. **Zoltán Thimár:** Writing - Review & Editing. **István Majzik:** Writing - Review & Editing. **Zoltán Micskei:** Writing - Review & Editing, Funding acquisition. **András Vörös:** Conceptualization, Writing - Review & Editing, Project administration.

References

- Baier C, Katoen J (2008) Principles of model checking. MIT Press
- Bengtsson J, Larsen K, Larsson F, et al (1995) UPPAAL—a tool suite for automatic verification of real-time systems. In: International hybrid systems workshop, Springer, pp 232–243, <https://doi.org/10.1007/BFb0020949>
- Bernardeschi C, Fantechi A, Gnesi S (2001) Formal validation of fault-tolerance mechanisms inside GUARDS. Reliab Eng Syst Saf 71(3):261–270. [https://doi.org/10.1016/S0951-8320\(00\)00078-8](https://doi.org/10.1016/S0951-8320(00)00078-8)
- Bernardeschi C, Fantechi A, Gnesi S (2002) Model checking fault tolerant systems. Softw Test Verification Reliab 12(4):251–275. <https://doi.org/10.1002/stvr.258>
- Bittner B, Bozzano M, Cavada R, et al (2016a) The xSAP safety analysis platform. In: Chechik M, Raskin J (eds) TACAS ’16, LNCS, vol 9636. Springer, pp 533–539, https://doi.org/10.1007/978-3-662-49674-9_31
- Bittner B, Bozzano M, Cimatti A, et al (2016b) Automated synthesis of timed failure propagation graphs. In: IJCAI, pp 972–978
- Bozzano M, Cimatti A, Fernandes Pires A, et al (2015a) Formal design and safety analysis of AIR6110 wheel brake system. In: CAV ’15, Springer, pp 518–535, https://doi.org/10.1007/978-3-319-21690-4_36
- Bozzano M, Cimatti A, Griggio A, et al (2015b) Efficient anytime techniques for model-based safety analysis. In: CAV ’15, LNCS, vol 9206. Springer, pp 603–621, https://doi.org/10.1007/978-3-319-21690-4_41
- Bozzano M, Munk P, Schweizer M, et al (2020) Model-based safety analysis of mode transitions. In: SAFECOMP ’20, Springer, pp 99–114, https://doi.org/10.1007/978-3-030-54549-9_7
- Cavada R, Cimatti A, Dorigatti M, et al (2014) The nuXmv symbolic model checker. In: CAV ’14, LNCS, vol 8559. Springer, pp 334–342, https://doi.org/10.1007/978-3-319-08867-9_22
- Cha SD, Son HS, Yoo J, et al (2003) Systematic evaluation of fault trees using real-time model checker UPPAAL. Reliab Eng Syst Saf 82(1):11–20. [https://doi.org/10.1016/S0951-8320\(03\)00059-0](https://doi.org/10.1016/S0951-8320(03)00059-0)

- Cimatti A, Clarke E, Giunchiglia E, et al (2002) NuSMV Version 2: An Open-Source Tool for Symbolic Model Checking. In: CAV '02, LNCS, vol 2404. Springer, Copenhagen, Denmark, https://doi.org/10.1007/3-540-45657-0_29
- Cimatti A, Griggio A, Mover S, et al (2014) IC3 modulo theories via implicit predicate abstraction. In: Ábrahám E, Havelund K (eds) Tools and Algorithms for the Construction and Analysis of Systems - 20th International Conference, TACAS 2014, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2014, Grenoble, France, April 5-13, 2014. Proceedings, Lecture Notes in Computer Science, vol 8413. Springer, pp 46–61, https://doi.org/10.1007/978-3-642-54862-8_4
- Clarke EM, Grumberg O, Jha S, et al (2000) Counterexample-guided abstraction refinement. In: Emerson EA, Sistla AP (eds) Computer Aided Verification, 12th International Conference, CAV 2000, Chicago, IL, USA, July 15-19, 2000, Proceedings, Lecture Notes in Computer Science, vol 1855. Springer, pp 154–169, https://doi.org/10.1007/10722167_15
- Elmqvist J, Nadjm-Tehrani S (2008) Tool support for incremental failure mode and effects analysis of component-based systems. In: DATE '08. ACM, pp 921–927, <https://doi.org/10.1109/DATE.2008.4484792>
- Emerson EA, Halpern JY (1986) “Sometimes” and “not never” revisited: on branching versus linear time temporal logic. *Journal of the ACM (JACM)* 33(1):151–178. <https://doi.org/10.1145/4904.4999>
- Gaaloul K, Menghi C, Nejati S, et al (2022) Combining genetic programming and model checking to generate environment assumptions. *IEEE Trans Software Eng* 48(9):3664–3685. <https://doi.org/10.1109/TSE.2021.3101818>
- Ghadhab M, Junges S, Katoen J, et al (2019) Safety analysis for vehicle guidance systems with dynamic fault trees. *Reliab Eng Syst Saf* 186:37–50. <https://doi.org/10.1016/j.ress.2019.02.005>
- Giannakopoulou D, Pasareanu CS, Barringer H (2002) Assumption generation for software component verification. In: 17th IEEE International Conference on Automated Software Engineering (ASE 2002), 23-27 September 2002, Edinburgh, Scotland, UK. IEEE Computer Society, pp 3–12, <https://doi.org/10.1109/ASE.2002.1114984>
- Gleirscher M, Marmsoler D (2020) Formal methods in dependable systems engineering: a survey of professionals from europe and north america. *Empir Softw Eng* 25(6):4473–4546. <https://doi.org/10.1007/S10664-020-09836-5>
- Gouyon D, Pétrin J, Cochard T, et al (2020) Architecture assessment for safety critical plant operation using reachability analysis of timed automata. *Reliab Eng Syst Saf* 199:106923. <https://doi.org/10.1016/j.ress.2020.106923>

- Graics B, Molnár V, Vörös A, et al (2020) Mixed-semantics composition of statecharts for the component-based design of reactive systems. *Software and Systems Modeling* 19(6):1483–1517. <https://doi.org/10.1007/s10270-020-00806-5>
- Graics B, Mondok M, Molnár V, et al (2025) Model-based testing of asynchronously communicating distributed controllers using validated mappings to formal representations. *Science of Computer Programming* p 103265. <https://doi.org/https://doi.org/10.1016/j.scico.2025.103265>
- Harel D, Thiagarajan P (2003) Message sequence charts. *UML for real: design of embedded real-time systems* pp 77–105. https://doi.org/10.1007/0-306-48738-1_4
- Heizmann M, Hoenicke J, Podelski A (2013) Software model checking for people who love automata. In: Sharygina N, Veith H (eds) *Computer Aided Verification - 25th International Conference, CAV 2013, Saint Petersburg, Russia, July 13-19, 2013. Proceedings, Lecture Notes in Computer Science*, vol 8044. Springer, pp 36–52, https://doi.org/10.1007/978-3-642-39799-8_2
- Henzinger TA, Jhala R, Majumdar R, et al (2002) Lazy abstraction. In: Launchbury J, Mitchell JC (eds) *Conference Record of POPL 2002: The 29th SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Portland, OR, USA, January 16-18, 2002*. ACM, pp 58–70, <https://doi.org/10.1145/503272.503279>
- Holzmann GJ (1997) The model checker SPIN. *IEEE Trans Software Eng* 23(5):279–295. <https://doi.org/10.1109/32.588521>
- Huang C, Li L (2020) Architectural design and analysis of a steer-by-wire system in view of functional safety concept. *Reliab Eng Syst Saf* 198:106822. <https://doi.org/10.1016/j.ress.2020.106822>
- Huang C, Naghdy F, Du H, et al (2019) Fault tolerant steer-by-wire systems: An overview. *Annual Reviews in Control* 47:98–111. <https://doi.org/10.1016/j.arcontrol.2019.04.001>
- ISO (2011) Road vehicles—functional safety. ISO 26262:2011, International Organization for Standardization, Geneva, Switzerland
- Jhala R, Majumdar R (2009) Software model checking. *ACM Comput Surv* 41(4):21:1–21:54. <https://doi.org/10.1145/1592434.1592438>
- Jimenez-Roa LA, Heskes T, Tinga T, et al (2023) Automatic inference of fault tree models via multi-objective evolutionary algorithms. *IEEE Trans Dependable Secur Comput* 20(4):3317–3327. <https://doi.org/10.1109/TDSC.2022.3203805>
- Kaleeswaran AP, Nordmann A, Vogel T, et al (2023) A user study for evaluation of formal verification results and their explanation at Bosch. *Empir Softw Eng* 28(5):125. <https://doi.org/10.1007/S10664-023-10353-4>

- Kang E, Enoiu EP, Marinescu R, et al (2013) A methodology for formal analysis and verification of EAST-ADL models. *Reliab Eng Syst Saf* 120:127–138. <https://doi.org/10.1016/j.res.2013.06.007>
- Kölbl M, Leue S (2018) Automated functional safety analysis of automated driving systems. In: Howar F, Barnat J (eds) *FMICS '18, LNCS*, vol 11119. Springer, pp 35–51, https://doi.org/10.1007/978-3-030-00244-2_3
- Lahtinen J, Valkonen J, Björkman K, et al (2012) Model checking of safety-critical software in the nuclear engineering domain. *Reliab Eng Syst Saf* 105:104–113. <https://doi.org/10.1016/j.res.2012.03.021>
- Lee EA, Sangiovanni-Vincentelli AL (2011) Component-based design for the future. In: 2011 Design, Automation & Test in Europe, IEEE, pp 1–5, <https://doi.org/10.1109/DATE.2011.5763168>
- Mader R, Armengaud E, Grießnig G, et al (2013) OASIS: an automotive analysis and safety engineering instrument. *Reliab Eng Syst Saf* 120:150–162. <https://doi.org/10.1016/j.res.2013.06.045>
- McMillan KL (2006) Lazy abstraction with interpolants. In: *CAV'06*, https://doi.org/10.1007/11817963_14
- Miller S, Anderson E, Wagner L, et al (2005) Formal verification of flight critical software. In: *AIAA Guidance, Navigation, and Control Conference and Exhibit*, p 6431, <https://doi.org/10.2514/6.2005-6431>
- Molnár V, Graics B, Vörös A, et al (2018) The Gamma Statechart Composition Framework: design, verification and code generation for component-based reactive systems. In: *ICSE '18*. ACM, pp 113–116, <https://doi.org/10.1145/3183440.3183489>
- Munir A, Koushanfar F (2020) Design and analysis of secure and dependable automotive CPS: A steer-by-wire case study. *IEEE Trans Dependable Secur Comput* 17(4):813–827. <https://doi.org/10.1109/TDSC.2018.2846741>
- Ni S, Zhuang Y, Cao Z, et al (2015) Modeling dependability features for real-time embedded systems. *IEEE Trans Dependable Secur Comput* 12(2):190–203. <https://doi.org/10.1109/TDSC.2014.2320714>
- Ortmeier F, Schellhorn G, Thums A, et al (2003) Safety analysis of the height control system for the elbtunnel. *Reliab Eng Syst Saf* 81(3):259–268. [https://doi.org/10.1016/S0951-8320\(03\)00090-5](https://doi.org/10.1016/S0951-8320(03)00090-5)
- Osaiweran A, Schuts M, Hooman J (2014) Experiences with incorporating formal techniques into industrial practice. *Empir Softw Eng* 19(4):1169–1194. <https://doi.org/10.1007/S10664-013-9251-2>

- Pakonen A, Buzhinsky I, Björkman K (2021) Model checking reveals design issues leading to spurious actuation of nuclear instrumentation and control systems. *Reliab Eng Syst Saf* 205:107237. <https://doi.org/10.1016/j.ress.2020.107237>
- Pontes RP, V  ras PC, Ambrosio AM, et al (2014) Contributions of model checking and cofi methodology to the development of space embedded software. *Empir Softw Eng* 19(1):39–68. <https://doi.org/10.1007/S10664-012-9215-Y>
- Poudel B, Munir A (2021) Design and evaluation of a reconfigurable ECU architecture for secure and dependable automotive cps. *IEEE Trans Dependable Secur Comput* 18(1):235–252. <https://doi.org/10.1109/TDSC.2018.2883057>
- Rupanov V, Buckl C, Fiege L, et al (2014) Employing early model-based safety evaluation to iteratively derive E/E architecture design. *Sci Comput Program* 90:161–179. <https://doi.org/10.1016/j.scico.2013.10.005>
- Rushby J (2002) Using model checking to help discover mode confusions and other automation surprises. *Reliab Eng Syst Saf* 75(2):167–177. [https://doi.org/10.1016/S0951-8320\(01\)00092-8](https://doi.org/10.1016/S0951-8320(01)00092-8)
- Sharvia S, Papadopoulos Y (2015) Integrating model checking with hip-hops in model-based safety analysis. *Reliab Eng Syst Saf* 135:64–80. <https://doi.org/10.1016/j.ress.2014.10.025>
- Stewart D, Liu J, Cofer DD, et al (2021) AADL-based safety analysis using formal methods applied to aircraft digital systems. *Reliab Eng Syst Saf* 213:107649. <https://doi.org/10.1016/j.ress.2021.107649>
- Szab   R, Szekeres D, Nagy SJ, et al (2023) Supplementary material for "aiding the design of critical software systems by iterative exploration of distinct requirement violation scenarios". Zenodo, <https://doi.org/10.5281/zenodo.10256749>
- Tonetta S (2009) Abstract model checking without computing the abstraction. In: Cavalcanti A, Dams D (eds) *FM 2009: Formal Methods, Second World Congress*, Eindhoven, The Netherlands, November 2-6, 2009. Proceedings, Lecture Notes in Computer Science, vol 5850. Springer, pp 89–105, https://doi.org/10.1007/978-3-642-05089-3_7
- T  th T, Hajdu A, V  r  s A, et al (2017) Theta: a framework for abstraction refinement-based model checking. In: *FMCAD 2017*, pp 176–179, <https://doi.org/10.23919/FMCAD.2017.8102257>